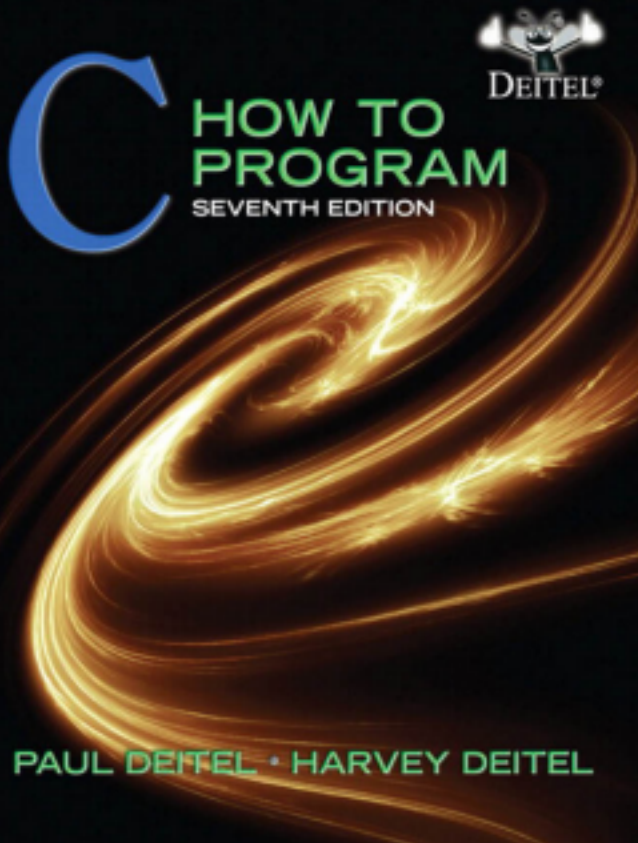




Deitel® Series Page

How To Program Series

Android How to Program
C++ How to Program, 8/E
C How to Program, 7/E
Java™ How to Program, 9/E
Java™ How to Program, Late Objects Version,
8/E Internet & World Wide Web How to
Program, 5/E Visual C++® 2008 How to
Program, 2/E Visual Basic® 2010 How to
Program
Visual C#® 2010 How to Program, 3/E

Simply Series

Simply C++: An App-Driven Tutorial
Approach Simply Java™ Programming: An
App-Driven Tutorial Approach
Simply C#: An App-Driven Tutorial
Approach Simply Visual Basic® 2010: An
App-Driven Approach, 4/E

CourseSmart Web Books

www.deitel.com/books/CourseSmart/C++
How to Program, 5/E, 6/E, 7/E & 8/E
Simply C++: An App-Driven Tutorial
Approach Java™ How to Program, 6/E, 7/E,
8/E & 9/E Simply Visual Basic 2010: An
App-Driven Approach, 4/E

(continued from previous column)

Visual Basic® 2010 How to Program
Visual Basic® 2008 How to Program
Visual C#® 2010 How to Program,
4/E Visual C#® 2008 How to
Program, 3/E

Deitel® Developer Series

AJAX, Rich Internet Applications and Web
Development for Programmers
Android for Programmers: An App-Driven
Approach
C++ for Programmers
C# 2010 for Programmers
iPhone® for Programmers: An App-Driven
Approach Java™ for Programmers, 2/e

JavaScript for Programmers

Fundamentals

Java™ Fundamentals

C# 2010 Fundamentals

iPhone® App Development Fundamentals

JavaScript Fundamentals

Visual Basic Fundamentals

LiveLessons Video Learning Products

www.deitel.com/books/LiveLessons/ Android

App Development Fundamentals C++

To receive updates on Deitel publications, Resource Centers, training courses, partner offers and more, please register for the free *Deitel® Buzz Online* e-mail newsletter at:

www.deitel.com/newsletter/subscribe.html

and join the Deitel communities on Twitter®

@deitel

Facebook®

facebook.com/DeitelFan

and Google+

[gplus.to/deitel](https://plus.to/deitel)

To communicate with the authors, send e-mail to:

deitel@deitel.com

For information on government and corporate *Dive-Into® Series* on-site seminars offered by Deitel & Associates, Inc. worldwide, visit:

www.deitel.com/training/

or write to

deitel@deitel.com

For continuing updates on Prentice Hall/Deitel publications visit:

www.deitel.com

www.pearsonhighered.com/deitel/

Visit the Deitel Resource Centers that will help you master programming languages, software development, Android and iPhone/iPad app development, and Internet- and web-related topics:

www.deitel.com/ResourceCenters.html

Paul Deitel
Deitel & Associates, Inc.

Harvey Deitel
Deitel & Associates,
Inc.

Editorial Director: *Marcia J. Horton*

Editor-in-Chief: *Michael Hirsch*

Associate Editor: *Carole Snyder*

Vice President, Marketing: *Patrice Jones*

Marketing Manager: *Yezan Alayan*

Marketing Coordinator: *Kathryn Ferranti*

Vice President, Production: *Vince O'Brien*

Managing Editor: *Jeff Holcomb*

Associate Managing Editor: *Robert Engelhardt*

Operations Specialist: *Lisa McDowell*

Art Director: *Anthony Gemmellaro*

Cover Design: *Paul Deitel, Harvey Deitel, Abbey Deitel, Anthony Gemmellaro*

Cover Photo Credit: Excellent backgrounds/Shutterstock.com

Media Editor: *Daniel Sandin*

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on page vi.

The authors and publisher of this book have used their best efforts in preparing this book. These efforts include the development, research, and testing of the theories and programs to determine their effectiveness. The authors and publisher make no warranty of any kind, expressed or implied, with regard to these programs or to the documentation contained in this book. The authors and publisher shall not be liable in any event for incidental or consequential damages in connection with, or arising out of, the furnishing, performance, or use of these programs.

Copyright © 2013, 2010, 2007, 2004, 2001 Pearson Education, Inc., publishing as Prentice Hall. All rights reserved. Manufactured in the United States of America. This publication is protected by Copyright, and permission should be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission(s) to use

material from this work, please submit a written request to Pearson Education, Inc., Permissions Department, One Lake Street, Upper Saddle River, New Jersey 07458, or you may fax your request to 201-236-3290.

Many of the designations by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Library of Congress Cataloging-in-Publication Data

Deitel, Paul J.

C : how to program / Paul Deitel, Deitel & Associates, Inc., Harvey Deitel, Deitel & Associates, Inc., Abbey Deitel, Deitel & Associates, Inc. -- Seventh edition.

pages cm -- (How to program series)

ISBN 978-0-13-299044-8

1. C (Computer program language) 2. C++ (Computer program language) 3. Java (Computer program language) I. Deitel, Harvey M., II. Deitel, Abbey. III. Title.

QA76.73.C15D44 2012

005.13'3--dc23

978-0-13-299044-8

2011051087

10 9 8 7 6 5 4 3 2 1 ISBN-10:

0-13-299044-X ISBN-13:

In Memory of Dennis Ritchie, creator of the C programming language and co-creator of the UNIX operating system.

Paul and Harvey Deitel

Trademarks

DEITEL, the double-thumbs-up bug and DIVE INTO are registered trademarks of Deitel and Associates, Inc.

MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THE INFORMATION CONTAINED IN THE DOCUMENTS AND RELATED GRAPHICS PUBLISHED AS PART OF THE SERVICES FOR ANY PURPOSE. ALL SUCH DOCUMENTS AND RELATED GRAPHICS ARE PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND. MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS HEREBY DISCLAIM ALL WARRANTIES AND CONDITIONS WITH REGARD TO THIS INFORMATION, INCLUDING ALL WARRANTIES AND CONDITIONS OF MERCHANTABILITY, WHETHER EXPRESS, IMPLIED OR STATUTORY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF INFORMATION AVAILABLE FROM THE SERVICES.

THE DOCUMENTS AND RELATED GRAPHICS CONTAINED HEREIN COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN. MICROSOFT AND/OR ITS

RESPECTIVE SUPPLIERS MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/ OR THE PROGRAM(S) DESCRIBED HEREIN AT ANY TIME. PARTIAL SCREEN SHOTS MAY BE VIEWED IN FULL WITHIN THE SOFTWARE VERSION SPECIFIED.

Appendices E through H are PDF documents posted online at the book's Companion Website (located at www.pearsonhighered.com/deitel).

Preface xix

1 Introduction to Computers, the Internet and the

Web 1

1.1 Introduction 2 1.2 Computers and the Internet in Industry and Research 2 1.3 Hardware and Software 5 1.3.1 Moore's Law 6 1.3.2 Computer Organization 6 1.4 Data Hierarchy 7 1.5 Programming Languages 9 1.6 The C Programming Language 10 1.7 C Standard Library 12 1.8 C++ and Other C-Based Languages 13 1.9 Object Technology 14 1.10 Typical C Program Development Environment 16 1.10.1 Phase 1: Creating a Program 16 1.10.2 Phases 2 and 3: Preprocessing and Compiling a C Program 16 1.10.3 Phase 4: Linking 18 1.10.4 Phase 5: Loading 18 1.10.5 Phase 6: Execution 18 1.10.6 Problems That May Occur at Execution Time 18 1.10.7 Standard Input, Standard Output and Standard Error Streams 18 1.11 Test-Driving a C Application in Windows, Linux and Mac OS X 19 1.11.1 Running a C Application from the Windows Command Prompt 20 1.11.2 Running a C Application Using GNU C with Linux 22 1.11.3 Running a C Application Using GNU C with Mac OS X 25 1.12 Operating Systems 27 1.12.1 Windows—A Proprietary Operating System 28 1.12.2 Linux—An Open-Source Operating System 28 1.12.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices 29 1.12.4 Google's Android 29

viii Contents

1.13 The Internet and World Wide Web 30 1.14 Some Key Software Development Terminology 31 1.15 Keeping Up-to-Date with Information Technologies 33 1.16 Web Resources 34

2 Introduction to C Programming 40

2.1 Introduction 41 2.2 A Simple C Program: Printing a Line of Text 41 2.3 Another Simple C Program: Adding Two Integers 45 2.4 Memory Concepts 49 2.5 Arithmetic in C 50 2.6 Decision Making: Equality and Relational Operators 54 2.7 Secure C Programming 58

3 Structured Program Development in C 70

3.1 Introduction 71 3.2 Algorithms 71 3.3 Pseudocode 71 3.4 Control Structures 72 3.5 The if Selection Statement 74 3.6 The if...else Selection Statement 75 3.7 The while Repetition Statement 79 3.8 Formulating Algorithms Case Study 1: Counter-Controlled Repetition 80 3.9 Formulating Algorithms with Top-Down, Stepwise Refinement

Case Study 2: Sentinel-Controlled Repetition 82 3.10 Formulating Algorithms
with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements 89
3.11 Assignment Operators 93 3.12 Increment and Decrement Operators 93 3.13
Secure C Programming 96

4 C Program Control 114 4.1 Introduction 115 4.2 Repetition
Essentials 115 4.3 Counter-Controlled Repetition 116 4.4 **for** Repetition Statement 117
4.5 **for** Statement: Notes and Observations 120 4.6 Examples Using the **for** Statement
121 4.7 **switch** Multiple-Selection Statement 124 4.8 **do...while** Repetition Statement 130
4.9 **break** and **continue** Statements 132 4.10 Logical Operators 134 4.11 Confusing
Equality (==) and Assignment (=) Operators 137

[Contents ix](#)

4.12 Structured Programming Summary 138 4.13 Secure C Programming 143

5 C Functions 158 5.1 Introduction 159 5.2 Program Modules in C 159
5.3 Math Library Functions 160 5.4 Functions 162 5.5 Function Definitions 162 5.6
Function Prototypes: A Deeper Look 166 5.7 Function Call Stack and Stack Frames
169 5.8 Headers 172 5.9 Passing Arguments By Value and By Reference 173 5.10
Random Number Generation 174 5.11 Example: A Game of Chance 179 5.12 Storage
Classes 182 5.13 Scope Rules 184 5.14 Recursion 187 5.15 Example Using Recursion:
Fibonacci Series 191 5.16 Recursion vs. Iteration 194 5.17 Secure C Programming 197

6 C Arrays 216 6.1 Introduction 217 6.2 Arrays 217 6.3 Defining Arrays
218 6.4 Array Examples 219 6.5 Passing Arrays to Functions 232 6.6 Sorting Arrays 236
6.7 Case Study: Computing Mean, Median and Mode Using Arrays 239 6.8 Searching
Arrays 244 6.9 Multidimensional Arrays 249 6.10 Variable-Length Arrays 256 6.11
Secure C Programming 259

7 C Pointers 277 7.1 Introduction 278 7.2 Pointer Variable Definitions and
Initialization 278 7.3 Pointer Operators 279 7.4 Passing Arguments to Functions by
Reference 282 7.5 Using the **const** Qualifier with Pointers 284 7.5.1 Converting a String
to Uppercase Using a Non-Constant
Pointer to Non-Constant Data 287

x Contents

7.5.2 Printing a String One Character at a Time Using a
Non-Constant Pointer to Constant Data 288 7.5.3 Attempting to
Modify a Constant Pointer to Non-Constant Data 290 7.5.4 Attempting to
Modify a Constant Pointer to Constant Data 291
7.6 Bubble Sort Using Pass-by-Reference 291 7.7 **sizeof** Operator 294 7.8 Pointer
Expressions and Pointer Arithmetic 297 7.9 Relationship between Pointers and Arrays
299 7.10 Arrays of Pointers 303 7.11 Case Study: Card Shuffling and Dealing Simulation
304 7.12 Pointers to Functions 309 7.13 Secure C Programming 314

8 C Characters and Strings 334

8.1 Introduction	335
8.2 Fundamentals of Strings and Characters	335
8.3 Character-Handling Library	337
8.3.1 Functions <code>isdigit</code> , <code>isalpha</code> , <code>isalnum</code> and <code>isxdigit</code>	338
8.3.2 Functions <code>islower</code> , <code>isupper</code> , <code>tolower</code> and <code>toupper</code>	340
8.3.3 Functions <code>isspace</code> , <code>isctrl</code> , <code>ispunct</code> , <code>isprint</code> and <code>isgraph</code>	341
8.4 String-Conversion Functions	342
8.4.1 Function <code>strtod</code>	343
8.4.2 Function <code>strtol</code>	344
8.4.3 Function <code>strtoul</code>	345
8.5 Standard Input/Output Library Functions	346
8.5.1 Functions <code>fgets</code> and <code>putchar</code>	346
8.5.2 Function <code>getchar</code>	348
8.5.3 Function <code>sprintf</code>	349
8.5.4 Function <code>sscanf</code>	349
8.6 String-Manipulation Functions of the String-Handling Library	350
8.6.1 Functions <code>strcpy</code> and <code>strncpy</code>	351
8.6.2 Functions <code>strcat</code> and <code>strncat</code>	352
8.7 Comparison Functions of the String-Handling Library	353
8.8 Search Functions of the String-Handling Library	354
8.8.1 Function <code>strchr</code>	355
8.8.2 Function <code>strcspn</code>	356
8.8.3 Function <code>strpbrk</code>	357
8.8.4 Function <code>strrchr</code>	357
8.8.5 Function <code>strspn</code>	358
8.8.6 Function <code>strstr</code>	358
8.8.7 Function <code>strtok</code>	359
8.9 Memory Functions of the String-Handling Library	360
8.9.1 Function <code>memcpy</code>	361
8.9.2 Function <code>memmove</code>	362
8.9.3 Function <code>memcmp</code>	363
8.9.4 Function <code>memchr</code>	363
8.9.5 Function <code>memset</code>	364
8.10 Other Functions of the String-Handling Library	365
8.10.1 Function <code>strerror</code>	365
8.10.2 Function <code>strlen</code>	365
8.11 Secure C Programming	366

[Contents xi](#)

9 C Formatted Input/Output 379

9.1 Introduction	380
9.2 Streams	380
9.3 Formatting Output with <code>printf</code>	380
9.4 Printing Integers	381
9.5 Printing Floating-Point Numbers	382
9.6 Printing Strings and Characters	384
9.7 Other Conversion Specifiers	385
9.8 Printing with Field Widths and Precision	386
9.9 Using Flags in the <code>printf</code> Format Control String	388
9.10 Printing Literals and Escape Sequences	391
9.11 Reading Formatted Input with <code>scanf</code>	391
9.12 Secure C Programming	398

10 C Structures, Unions, Bit Manipulation and Enumerations 405

10.1 Introduction	406
10.2 Structure Definitions	406
10.2.1 Self-Referential Structures	407
10.2.2 Defining Variables of Structure Types	407
10.2.3 Structure Tag Names	408
10.2.4 Operations That Can Be Performed on Structures	408
10.3 Initializing Structures	409
10.4 Accessing Structure Members	409
10.5 Using Structures with Functions	411
10.6 <code>typedef</code>	411
10.7 Example: High-Performance Card Shuffling and Dealing Simulation	412
10.8 Unions	415
10.8.1 Union Declarations	415
10.8.2 Operations That Can Be Performed on Unions	415
10.8.3 Initializing Unions in Declarations	416
10.8.4 Demonstrating Unions	416
10.9 Bitwise Operators	417
10.9.1 Displaying an Unsigned Integer in Bits	418
10.9.2 Making Function <code>displayBits</code> More Scalable and Portable	420
10.9.3 Using the Bitwise AND, Inclusive OR, Exclusive OR and Complement Operators	420

10.9.4 Using the Bitwise Left- and Right-Shift Operators 423 10.9.5 Bitwise Assignment Operators 425 10.10 Bit Fields 426 10.11 Enumeration Constants 429 10.12 Secure C Programming 431

11 C File Processing 441 11.1 Introduction 442 11.2 Files and Streams 442 11.3 Creating a Sequential-Access File 443 11.4 Reading Data from a Sequential-Access File 448 11.5 Random-Access Files 452 11.6 Creating a Random-Access File 453 11.7 Writing Data Randomly to a Random-Access File 455 11.8 Reading Data from a Random-Access File 458 11.9 Case Study: Transaction-Processing Program 459 11.10 Secure C Programming 465

12 C Data Structures 476 12.1 Introduction 477 12.2 Self-Referential Structures 478 12.3 Dynamic Memory Allocation 478 12.4 Linked Lists 479
12.4.1 Function `insert` 485 12.4.2 Function `delete` 487 12.4.3 Function `printList` 488
12.5 Stacks 488 12.5.1 Function `push` 492 12.5.2 Function `pop` 492 12.5.3 Applications of Stacks 493
12.6 Queues 494 12.6.1 Function `enqueue` 498 12.6.2 Function `dequeue` 499
12.7 Trees 500 12.7.1 Function `insertNode` 504 12.7.2 Traversals: Functions `inOrder`, `preOrder` and `postOrder` 504 12.7.3 Duplicate Elimination 505 12.7.4 Binary Tree Search 505 12.7.5 Other Binary Tree Operations 505 12.8 Secure C Programming 506

13 C Preprocessor 517 13.1 Introduction 518 13.2 `#include` Preprocessor Directive 518

[Contents](#) xiii

13.3 `#define` Preprocessor Directive: Symbolic Constants 519 13.4 `#define` Preprocessor Directive: Macros 519 13.5 Conditional Compilation 521 13.6 `#error` and `#pragma` Preprocessor Directives 522 13.7 `#` and `##` Operators 523 13.8 Line Numbers 523 13.9 Predefined Symbolic Constants 523 13.10 Assertions 524 13.11 Secure C Programming 524

14 Other C Topics 529 14.1 Introduction 530 14.2 Redirecting I/O 530 14.3 Variable-Length Argument Lists 531 14.4 Using Command-Line Arguments 533 14.5 Notes on Compiling Multiple-Source-File Programs 534 14.6 Program Termination with `exit` and `atexit` 536 14.7 Suffixes for Integer and Floating-Point Literals 537 14.8 Signal Handling 538 14.9 Dynamic Memory Allocation: Functions `calloc` and `realloc` 540 14.10 Unconditional Branching with `goto` 541

15 C++ as a Better C; Introducing Object Technology 547 15.1 Introduction 548 15.2 C++ 548 15.3 A Simple Program: Adding Two Integers 549 15.4 C++ Standard Library 551 15.5 Header Files 552 15.6 Inline Functions 554 15.7 References and Reference Parameters 556 15.8 Empty Parameter Lists 561 15.9 Default Arguments 561 15.10 Unary Scope Resolution

Operator 563 15.11 Function Overloading 564 15.12 Function Templates 567 15.13 Introduction to C++ Standard Library Class Template `vector` 570 15.14 Introduction to Object Technology and the UML 576 15.15 Wrap-Up 579

16 Introduction to Classes, Objects and Strings

586 16.1 Introduction 587 16.2 Defining a Class with a Member Function 587

xiv [Contents](#)

16.3 Defining a Member Function with a Parameter 590 16.4 Data Members, *set* Functions and *get* Functions 593 16.5 Initializing Objects with Constructors 599 16.6 Placing a Class in a Separate File for Reusability 603 16.7 Separating Interface from Implementation 606 16.8 Validating Data with *set* Functions 612 16.9 Wrap-Up 617

17 Classes: A Deeper Look, Part 1 623

17.1 Introduction 624 17.2 Time Class Case Study 625 17.3 Class Scope and Accessing Class Members 632 17.4 Separating Interface from Implementation 633 17.5 Access Functions and Utility Functions 634 17.6 Time Class Case Study: Constructors with Default Arguments 637 17.7 Destructors 642 17.8 When Constructors and Destructors Are Called 643 17.9 Time Class Case Study: A Subtle Trap—Returning a Reference to
a `private` Data Member 646 17.10 Default Memberwise Assignment 649 17.11 Wrap-Up 652

18 Classes: A Deeper Look, Part 2 658

18.1 Introduction 659 18.2 `const` (Constant) Objects and `const` Member Functions 659 18.3 Composition: Objects as Members of Classes 667 18.4 `friend` Functions and `friend` Classes 673 18.5 Using the `this` Pointer 675 18.6 `static` Class Members 680 18.7 Proxy Classes 685 18.8 Wrap-Up 689

19 Operator Overloading; Class `string` 695

19.1 Introduction 696 19.2 Using the Overloaded Operators of Standard Library Class `string` 697 19.3 Fundamentals of Operator Overloading 700 19.4 Overloading Binary Operators 701 19.5 Overloading the Binary Stream Insertion and Stream Extraction Operators 702 19.6 Overloading Unary Operators 706 19.7 Overloading the Unary Prefix and Postfix `++` and `--` Operators 707 19.8 Case Study: A `Date` Class 708 19.9 Dynamic Memory Management 713

[Contents](#) xv

19.10 Case Study: `Array` Class 715 19.10.1 Using the `Array` Class 716 19.10.2 `Array` Class Definition 719 19.11 Operators as Member Functions vs. Non-Member Functions 727 19.12 Converting between Types 727 19.13 `explicit` Constructors 729 19.14 Building a `String` Class 731 19.15 Wrap-Up 732

20 Object-Oriented Programming: Inheritance

743 20.1 Introduction 744 20.2 Base Classes and Derived Classes 744 20.3 **protected** Members 747 20.4 Relationship between Base Classes and Derived Classes 747
 20.4.1 Creating and Using a **CommissionEmployee** Class 748 20.4.2 Creating a **BasePlusCommissionEmployee** Class Without
 Using Inheritance 752 20.4.3 Creating a **CommissionEmployee–BasePlusCommissionEmployee** Inheritance Hierarchy 758 20.4.4 **CommissionEmployee–BasePlusCommissionEmployee** Inheritance Hierarchy Using **protected** Data 763 20.4.5 **CommissionEmployee–BasePlusCommissionEmployee** Inheritance Hierarchy Using **private** Data 766 20.5 Constructors and Destructors in Derived Classes 771 20.6 **public**, **protected** and **private** Inheritance 771 20.7 Software Engineering with Inheritance 772 20.8 Wrap-Up 773

21 Object-Oriented Programming: Polymorphism

778 21.1 Introduction 779 21.2 Introduction to Polymorphism: Polymorphic Video Game 780 21.3 Relationships Among Objects in an Inheritance Hierarchy 780
 21.3.1 Invoking Base-Class Functions from Derived-Class Objects 781 21.3.2 Aiming Derived-Class Pointers at Base-Class Objects 784 21.3.3 Derived-Class Member-Function Calls via Base-Class Pointers 785 21.3.4 Virtual Functions 787 21.4 Type Fields and **switch** Statements 793 21.5 Abstract Classes and Pure virtual Functions 793 21.6 Case Study: Payroll System Using Polymorphism 795
 21.6.1 Creating Abstract Base Class **Employee** 796 21.6.2 Creating Concrete Derived Class **SalariedEmployee** 800 21.6.3 Creating Concrete Derived Class **CommissionEmployee** 802 21.6.4 Creating Indirect Concrete Derived Class **BasePlusCommissionEmployee** 804 21.6.5 Demonstrating Polymorphic Processing 806

xvi **Contents**

21.7 (Optional) Polymorphism, Virtual Functions and Dynamic Binding “Under the Hood” 810 21.8 Case Study: Payroll System Using Polymorphism and Runtime Type Information with Downcasting, **dynamic_cast**, **typeid** and **type_info** 813 21.9 Virtual Destructors 817 21.10 Wrap-Up 817

22 Templates 823 22.1 Introduction 824 22.2 Function Templates 824 22.3 Overloading Function Templates 827 22.4 Class Templates 828 22.5 Nontype Parameters and Default Types for Class Templates 834 22.6 Wrap-Up 835

23 Stream Input/Output 839 23.1 Introduction 840 23.2 Streams 841

23.2.1 Classic Streams vs. Standard Streams 841 23.2.2 **iostream** Library Headers 842 23.2.3 Stream Input/Output Classes and Objects 842 23.3 Stream Output 845 23.3.1 Output of **char *** Variables 845 23.3.2 Character Output Using Member Function **put** 845 23.4 Stream Input 846 23.4.1 **get** and **getline** Member Functions 846 23.4.2 **istream** Member Functions **peek**, **putback** and **ignore** 849 23.4.3 Type-Safe I/O 849 23.5 Unformatted I/O Using **read**, **write** and **gcount** 849 23.6 Introduction to Stream Manipulators 850 23.6.1 Integral Stream Base: **dec**, **oct**, **hex** and **setbase** 851 23.6.2 Floating-Point Precision (**precision**, **setprecision**) 851 23.6.3 Field Width (**width**, **setw**) 853 23.6.4 User-Defined Output Stream Manipulators 854 23.7 Stream Format States

and Stream Manipulators 856 23.7.1 Trailing Zeros and Decimal Points (**showpoint**) 856
23.7.2 Justification (**left**, **right** and **internal**) 857 23.7.3 Padding (**fill**, **setfill**) 859 23.7.4
Integral Stream Base (**dec**, **oct**, **hex**, **showbase**) 860 23.7.5 Floating-Point Numbers;
Scientific and Fixed Notation
(**scientific**, **fixed**) 861 23.7.6 Uppercase/Lowercase Control (**uppercase**)
862 23.7.7 Specifying Boolean Format (**boolalpha**) 862 23.7.8 Setting and
Resetting the Format State via Member

Function flags 863

[Contents](#) xvii

23.8 Stream Error States 864 23.9 Tying an Output Stream to an Input Stream 866
23.10 Wrap-Up 867

24 Exception Handling: A Deeper Look 876 24.1

Introduction 877 24.2 Example: Handling an Attempt to Divide by Zero 877 24.3
When to Use Exception Handling 883 24.4 Rethrowing an Exception 884 24.5
Processing Unexpected Exceptions 885 24.6 Stack Unwinding 886 24.7 Constructors,
Destructors and Exception Handling 888 24.8 Exceptions and Inheritance 888 24.9
Processing new Failures 889 24.10 Class `unique_ptr` and Dynamic Memory Allocation
892 24.11 Standard Library Exception Hierarchy 894 24.12 Wrap-Up 896

A Operator Precedence Charts 902 B ASCII

Character Set 906

C Number Systems 907 C.1 Introduction 908 C.2 Abbreviating

Binary Numbers as Octal and Hexadecimal Numbers 911 C.3 Converting Octal and
Hexadecimal Numbers to Binary Numbers 912 C.4 Converting from Binary, Octal or
Hexadecimal to Decimal 912 C.5 Converting from Decimal to Binary, Octal or
Hexadecimal 913 C.6 Negative Binary Numbers: Two's Complement Notation 915

D Game Programming: Solving Sudoku 920 D.1

Introduction 920 D.2 Deitel Sudoku Resource Center 921 D.3 Solution Strategies 921
D.4 Programming Sudoku Puzzle Solvers 925 D.5 Generating New Sudoku Puzzles 926
D.6 Conclusion 928

Appendices on the Web 929 Index 930

xviii [Contents](#)

Appendices E through H are PDF documents posted online at the book's
Companion Website (located at www.pearsonhighered.com/deitel).

E Sorting: A Deeper Look

F Introduction to the New C Standard

G Using the Visual Studio Debugger

H Using the GNU Debugger

Welcome to the C programming language—and to C++, too! This book presents leading edge computing technologies for college students, instructors and software development professionals.

At the heart of the book is the Deitel signature “live-code approach.” We present concepts in the context of complete working programs, rather than in code snippets. Each code example is followed by one or more sample executions. Read the online Before You Begin section (www.deitel.com/books/cht7/cht7_BYB.pdf) to learn how to set up your computer to run the hundreds of code examples. All the source code is available at www.deitel.com/books/cht7/ and www.pearsonhighered.com/deitel. Use the source code we provide to *run every program* as you study it.

We believe that this book and its support materials will give you an informative, challenging and entertaining introduction to C. As you read the book, if you have questions, send an e-mail to deitel@deitel.com—we’ll respond promptly. For book updates, visit www.deitel.com/books/cht7/, join our communities on Facebook (www.deitel.com/deitelfan), Twitter (@deitel) and Google+ (gplus.to/deitel), and subscribe to the *Deitel® Buzz Online* newsletter (www.deitel.com/newsletter/subscribe.html).

New and Updated Features

Here are some key features of *C How to Program, 7/e*:

- **Coverage of the New C standard.** The previous edition of the book conformed to “standard C” and included a detailed appendix on the C99 standard. The New C Standard was approved just before *C How to Program, 7/e* went to publication. The new standard incorporates both C99 and the more recent C1X—now referred to as C11 or simply “the C standard” since its approval in 2011. Support for the new standard varies by compiler. The vast majority of our readership uses either the GNU gcc compiler—which supports several of the key features in the new standard—or the Microsoft Visual C++ compiler. Microsoft supports only a limited subset of the features that were added to C in C99 and C11—primarily the features that are also required by the C++ standard. To accommodate all of our readers, we placed the discussion of the new standard in optional, easy-to-use or-omit sections and in Appendix F, Introduction to the New C Standard. We’ve also replaced various deprecated capabilities with newer preferred versions as a result of the new C standard.
- **New Chapter 1.** The new Chapter 1 engages students with intriguing facts and figures to get them excited about studying computers and computer programming. The chapter includes a table of some of the research made possible by computers and the Internet, current technology trends and hardware discussion, the data hierarchy, a new section on social networking, a

lications and websites that will help you stay up to date with the latest technology news and trends, and updated exercises. We've included test-drives that show how to run a command-line C program on Microsoft Windows, Linux and Mac OS X.

- **Secure C Programming Sections.** We've added notes about secure C programming to many of the C programming chapters. We've also posted a Secure C Programming Resource Center at www.deitel.com/SecureC/. For more details, see the section "A Note About Secure C Programming" in this Preface.
- **Focus on Performance Issues.** C (and C++) are favored by designers of performance-intensive applications such as operating systems, real-time systems, embedded systems and communications systems, so we focus intensively on performance issues.
- **"Making a Difference" Exercise Sets.** We encourage you to use computers and the Internet to research and solve problems that really matter. These exercises are meant to increase awareness of important issues the world is facing. We hope you'll approach them with your own values, politics and beliefs.
- **All Code Tested on Windows and Linux.** We've tested every example and exercise program using Visual C++ and GNU gcc in Windows and Linux, respectively.
- **Updated Coverage of C++ and Object-Oriented Programming.** We updated Chapters 15–24 on object-oriented programming in C++ with material from our textbook *C++ How to Program, 8/e*.
- **Sorting: A Deeper Look.** Sorting places data in order, based on one or more sort keys. We begin our presentation of sorting with a simple algorithm in Chapter 6— in Appendix E, we present a deeper look. We consider several algorithms and compare them with regard to their memory consumption and processor demands. For this purpose, we introduce Big O notation, which indicates how hard an algorithm may have to work to solve a problem. Through examples and exercises, Appendix E discusses the selection sort, insertion sort, recursive merge sort, recursive selection sort, bucket sort and recursive Quicksort. Sorting is an interesting problem because different sorting techniques achieve the same final result but they can vary hugely in their consumption of memory, CPU time and other system resources.
- **Titled Programming Exercises.** All the programming exercises are titled to help instructors tune assignments for their classes.
- **Debugger Appendices.** We've updated the Visual C++[®] and GNU gdb debugging appendices.
- **Order of Evaluation.** We added cautions about order of evaluation issues.
- **Additional Exercises.** We added more function pointer exercises. We also added a Fibonacci exercise project that improves the Fibonacci recursion example (tail recursion).
- **C++-Style // Comments.** We use the newer, more concise C++-style `//` comments in preference to C's older style `/*...*/` comments.
- **C Standard Library.** Section 1.7 references P.J. Plauger's Dinkumware website (www.dinkumware.com/manuals/default.aspx) where students can find thor

A Note About Secure C Programming

Throughout this book, we focus on C programming *fundamentals*. When we write each *How to Program* book, we search the corresponding language's standards document for the features that we feel novices need to learn in a first programming course, and features that existing programmers need to know to *begin* working in that language. We must also cover programming fundamentals *and* computer-science fundamentals for novice program mers—our core audience.

Industrial-strength coding techniques in any programming language are beyond the scope of an introductory textbook. For that reason, our Secure C Programming sections present some key issues and techniques, and provide links and references so you can continue learning.

Experience has shown that it's difficult to build industrial-strength systems that stand up to attacks from viruses, worms, etc. Today, via the Internet, such attacks can be instantaneous and global in scope. Software vulnerabilities often come from simple programming issues. Building security into software from the start of the development cycle can greatly reduce costs and vulnerabilities.

The CERT[®] Coordination Center (www.cert.org) was created to analyze and respond promptly to attacks. CERT—the Computer Emergency Response Team—publishes and promotes secure coding standards to help C programmers and others implement industrial-strength systems that avoid the programming practices that open systems to attacks. The CERT standards evolve as new security issues arise.

We've upgraded our code (as appropriate for an introductory book) to conform to various CERT recommendations. If you'll be building C systems in industry, consider reading *The CERT C Secure Coding Standard* (Robert Seacord, Addison-Wesley Professional, 2009) and *Secure Coding in C and C++* (Robert Seacord, Addison-Wesley Professional, 2006). The CERT guidelines are available free online at www.securecoding.cert.org. Mr. Seacord, a technical reviewer for the C portion of this book, provided specific recommendations on each of our new Secure C Programming sections. Mr. Seacord is the Secure Coding Manager at CERT at Carnegie Mellon University's Software Engineering Institute (SEI) and an adjunct professor in the Carnegie Mellon University School of Computer Science.

The Secure C Programming sections at the ends of Chapters 2–13 discuss many important topics, including testing for arithmetic overflows, using unsigned integer types, new more secure functions in the C standard's Annex K, the importance of checking the status information returned by standard-library functions, range checking, secure random-number generation, array bounds checking, techniques for preventing buffer overflows, input validation, avoiding undefined behaviors, choosing functions that return status information vs. using similar functions that do not, ensuring that pointers are always NULL or contain valid addresses, using C functions vs. using preprocessor macros, and more.

Web-Based Materials

This book is supported by substantial online materials. The book's Companion Website (www.pearsonhighered.com/deitel) contains source code for all the code examples and the following appendices in *searchable* PDF format:

- Appendix E, Sorting: A Deeper Look

- Appendix G, Using the Visual Studio Debugger
- Appendix H, Using the GNU Debugger

Dependency Charts

Figures 1 and 2 show the dependencies among the chapters to help instructors plan their syllabi. *C How to Program, 7/e* is appropriate for CS1 and CS2 courses, and intermediate level C and C++ programming courses. The C++ part of the book assumes that you've studied the C part.

C Chapter Dependency Chart

[Note: Arrows pointing into a chapter indicate that chapter's dependencies.]

5 C Functions

6 C Arrays

Pointers and Strings 7 C

Pointers

8 C Characters and Strings

Aggregate Types

10 C Structures, Unions, Bit
Manipulations and
Enumerations

Streams and Files 9 C

Formatted Input/Output

11 C File Processing

Introduction

1 Introduction to Computers,
the Internet and the Web

Data Structures

Intro to Programming 2

Intro to C Programming

**Control Statements,
Functions and Arrays** 3

Structured Program
Development in C

4 C Program Control

**Other Topics and the New C
Standard**

5.14–5.16 Recursion 12 C Data Structures

Fig. 1 | C chapter dependency chart.**C++ Chapter
Dependency
Chart**

[Note: Arrows pointing into a chapter indicate that chapter's dependencies.]

**Object-Based
Programming**

15 C++ as a Better C;
Intro to Object Technology 16

Intro to Classes and Objects

17 Classes: A Deeper
Polymorphism

Look, Part 1

18 Classes: A Deeper
Look, Part 2

19 Operator Overloading

**Object-Oriented
Programming**

20 OOP: Inheritance

Teaching Approach xxiii

Fig. 2 | C++ chapter

dependency chart.

Teaching**Approach**

23 Stream Input/Output
24 Exception Handling

21 OOP: 22 Templates

C How to Program, 7/e, contains a rich collection of examples. We focus on good software engineering and stressing program clarity.

Syntax Shading. For readability, we syntax shade the code, similar to the way most IDEs and code editors syntax color code. Our syntax-shading conventions are:

comments appear like this
keywords appear like this
constants and literal values appear like this
all other code appears in black

Code Highlighting. We place gray rectangles around the key code.

Using Fonts for Emphasis. We place the key terms and the index's page reference for each defining occurrence in **bold blue** text for easy reference. We emphasize on-screen components in the **bold Helvetica** font (e.g., the File menu) and C program text in the

Lucida font (for example, `int x = 5;`).

Objectives. The opening quotes are followed by a list of chapter objectives.

Illustrations/Figures. Abundant charts, tables, line drawings, UML diagrams, programs and program output are included.

Programming Tips. We include programming tips to help you focus on important aspects of program development. These tips and practices represent the best we've gleaned from a combined seven decades of programming and teaching experience.

xxiv [Preface](#)



Good Programming Practices

The Good Programming Practices call attention to techniques that will help you produce programs that are clearer, more understandable and more maintainable.



Common Programming Errors

Pointing out these Common Programming Errors reduces the likelihood that you'll make them.



Error-Prevention Tips

These tips contain suggestions for exposing and removing bugs from your programs; many describe aspects of C that prevent bugs from getting into programs in the first place.



Performance Tips

These tips highlight opportunities for making your programs run faster or minimizing the amount of memory that they occupy.



Portability Tips

The Portability Tips help you write code that will run on a variety of platforms.



Software Engineering Observations

The Software Engineering Observations highlight architectural and design issues that affect the construction of software systems, especially large-scale systems.

Summary Bullets. We present a section-by-section, bullet-list summary of the chapter.

Terminology. We include an alphabetized list of the important terms defined in each chapter with the page number of each term's defining occurrence for easy reference.

Self-Review Exercises and Answers. Extensive self-review exercises and answers are included for self-study.

Exercises. Each chapter concludes with a substantial set of exercises

- including: • simple recall of important terminology and concepts
- identifying the errors in code samples
- writing individual program statements
- writing small portions of C functions and C++ member functions and classes • writing complete programs
- implementing major projects

Index. We've included an extensive index, which is especially useful when you use the book as a reference. Defining occurrences of key terms are highlighted with a **bold blue**

Software Used in *C How to Program, 7/e*

We wrote *C How to Program, 7/e* using Microsoft's free Visual C++ Express Edition (which can compile both C and C++ programs and can be downloaded from [www.microsoft.com/](http://www.microsoft.com/express/downloads/)

C++ IDE Resource Kit xxv

[express/downloads/](http://www.microsoft.com/express/downloads/)) and the free GNU C and C++ compilers (gcc.gnu.org/install/binaries.html), which are already installed on most Linux systems and can be installed on Mac OS X and Windows systems. Apple includes GNU C and C++ in their Xcode development tools, which Mac OS X users can download from developer.apple.com/technologies/tools/xcode.html.

For other free C and C++ compilers, visit:

www.thefreecountry.com/compilers/cpp.shtml
www.compilers.net/Dir/Compilers/CCpp.htm
www.freebyte.com/programming/cpp/#cppcompilers
en.wikipedia.org/wiki/List_of_compilers#C.2B.2B_compilers

C++ IDE Resource Kit

Your instructor may have ordered through your college bookstore a Value Pack edition of *C How to Program, 7/e* that comes bundled with the C++ IDE Resource Kit—most C++ compilers also support C. This kit contains CD or DVD versions of:

- Microsoft® Visual Studio 2010 Express Edition (www.microsoft.com/express/) • Dev C++ (www.bloodshed.net/download.html)
- NetBeans (netbeans.org/downloads/index.html)
- Eclipse (eclipse.org/downloads/)
- CodeLite (codelite.org/LiteEditor/Download)

You can also download these software packages from the websites specified above. The C++ IDE Resource Kit also includes access to a Companion Website containing step-by-step written instructions and VideoNotes to help you get started with each development environment. If your book did not come with the C++ IDE Resource Kit, you can purchase access to the Resource Kit's Companion Website from www.pearsonhighered.com/cppidekit/.

CourseSmart Web Books

Today's students and instructors have increasing demands on their time and money. Pearson has responded to that need by offering digital texts and course materials online through CourseSmart. CourseSmart allows faculty to review course materials online, saving time and costs. It offers students a high-quality digital version of the text for less than the cost of a print copy. Students receive the same content offered in the print textbook enhanced by search, note-taking and printing tools. For more information, visit www.coursesmart.com.

Instructor Resources

The following supplements are available to *qualified instructors only* through Pearson

- **PowerPoint® slides** containing all the code and figures in the text, plus bulleted items that summarize key points.
 - **Test Item File** of multiple-choice questions (approximately two per book section)
- xxvi **Preface**

- **Solutions Manual** with solutions to most of the end-of-chapter exercises. Please check the Instructor Resource Center to determine which exercises have solutions.

Please do not write to us requesting access to the Pearson Instructor's Resource Center. Access is restricted to college instructors teaching from the book. Instructors may obtain access only through their Pearson representatives. If you're not a registered faculty member, contact your Pearson representative or visit www.pearsonhighered.com/educator/relocator/.

Solutions are *not* provided for “project” exercises. Check out our Programming Projects Resource Center for lots of additional exercise and project possibilities (www.deitel.com/ProgrammingProjects/).

Acknowledgments

We'd like to thank Abbey Deitel and Barbara Deitel for long hours devoted to this project. We're fortunate to have worked with the dedicated team of publishing professionals at Pearson. We appreciate the guidance, savvy and energy of Michael Hirsch, Editor-in Chief of Computer Science. Carole Snyder and Bob Engelhardt did a marvelous job managing the review and production processes, respectively.

C How to Program, 7/e Reviewers

We wish to acknowledge the efforts of our reviewers. Under tight deadlines, they scrutinized the text and the programs and provided countless suggestions for improving the presentation: Dr. John F. Doyle (Indiana University Southeast), Hemanth H.M. (Software Engineer at SonicWALL), Vytautas Leonavicius (Microsoft), Robert Seacord (Secure Coding Manager at SEI/CERT, author of *The CERT C Secure Coding Standard* and technical expert for the international standardization working group for the programming language C) and José Antonio González Seco (Parliament of Andalusia).

Other Recent Editions Reviewers

William Albrecht (University of South Florida), Ian Barland (Radford University), Ed James Beckham (Altera), John Benito (Blue Pilot Consulting, Inc. and Convener of ISO WG14—the Working Group responsible for the C Programming Language Standard), Alireza Fazelpour (Palm Beach Community College), Mahesh Hariharan (Microsoft), Kevin Mark Jones (Hewlett Packard), Lawrence Jones (UGS Corp.), Don Kostuch (Independent Consultant), Xiaolong Li (Indiana State University), William Mike Miller (Edison Design Group, Inc.), Tom Rethard (The University of Texas at Arlington), Benjamin Seyfarth (University of Southern Mississippi), Gary Sibbitts (St. Louis Community College at Meramec), William Smith (Tulsa Community College) and Douglas Walls (Senior Staff Engineer, C compiler, Sun Microsystems).

Well, there you have it! C is a powerful programming language that will help you write high-performance programs quickly and effectively. C scales nicely into the realm of enterprise systems development to help organizations build their business-critical and mission-critical information systems. As you read the book, we would sincerely

appreciate your comments, criticisms, corrections and suggestions for improving the text. Please address all correspondence to:

deitel@deitel.com

About the Authors xxvii

We'll respond promptly, and post corrections and clarifications on:

www.deitel.com/books/cht7/

We hope you enjoy working with *C How to Program, Seventh Edition* as much as we enjoyed writing it!

Paul Deitel

Harvey Deitel

January 2012

About the Authors

Paul Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., is a graduate of MIT, where he studied Information Technology. Through Deitel & Associates, Inc., he has delivered hundreds of programming courses to industry clients, including Cisco, IBM, Siemens, Sun Microsystems, Dell, Lucent Technologies, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, White Sands Missile Range, Rogue Wave Software, Boeing, SunGard Higher Education, Stratus, Cambridge Technology Partners, One Wave, Hyperion Software, Adra Systems, Entergy, CableData Systems, Nortel Networks, Puma, iRobot, Invensys and many more. He and his co-author, Dr. Harvey M. Deitel, are the world's best-selling programming-language textbook/professional book/video authors.

Dr. Harvey Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has 50 years of experience in the computer field. Dr. Deitel earned B.S. and M.S. degrees from MIT and a Ph.D. from Boston University. He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., in 1991 with his son, Paul Deitel. The Deitels' publications have earned international recognition, with translations published in Chinese, Korean, Japanese, German, Russian, Spanish, French, Polish, Italian, Portuguese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of professional programming seminars to major corporations, academic institutions, government organizations and the military.

Corporate Training from Deitel & Associates, Inc. Deitel & Associates, Inc., founded by Paul Deitel and Harvey Deitel, is an internationally recognized authoring, corporate training and software development organization specializing in computer programming languages, object technology, Android and iPhone app development and Internet and web software technology. The company offers instructor led training courses delivered at client sites worldwide on major programming languages and platforms, including C, C++, Visual C++[®], Java[™], Visual C#[®], Visual Basic[®], XML[®], Python[®], object technology, Internet and web programming, Android app development, Objective-C and iPhone app development and a growing list of additional programming and software development courses. The company's clients include many of the world's largest companies, government agencies, branches of the military, and academic institutions.

Through its 36-year publishing partnership with Prentice Hall/Pearson, Deitel &

books and *LiveLessons* video courses. Deitel & Associates, Inc. and the authors can be reached at:

deitel@deitel.com

To learn more about Deitel's *Dive Into*® *Series* Corporate Training curriculum, visit:

www.deitel.com/training/

To request a proposal for worldwide on-site, instructor-led training at your company or organization, e-mail deitel@deitel.com.

Individuals wishing to purchase Deitel books and *LiveLessons* video training can do so through www.deitel.com. Bulk orders by corporations, the government, the military and academic institutions should be placed directly with Pearson. For more information, visit www.pearsoned.com/professional/index.htm.

Introduction to Computers, the Internet and the Web

*The chief merit of language is
clearness.*

—Galen

*Our life is frittered away by
detail. ... Simplify, simplify.*

—Henry David Thoreau

*He had a wonderful talent for
packing thought close, and
rendering it portable.*

—Thomas B. Macaulay

*Man is still the most
extraordinary computer of all.*

—John F. Kennedy

Objectives

In this chapter, you'll learn:

- Basic computer concepts.
- The different types of

programming languages.

- The history of the C programming language.
- The purpose of the C Standard Library.
- The elements of a typical C program development environment.
- To test-drive a C application in Windows, Linux and Mac OS X.
- Some basics of the Internet and the World Wide Web.

2 Chapter 1 Introduction to Computers, the Internet and the Web

1.1 Introduction

1.2 Computers and the Internet in Industry and Research

1.3 Hardware and Software

1.3.1 Moore's Law

1.3.2 Computer Organization

1.4 Data Hierarchy

1.5 Programming Languages

1.6 The C Programming

Language 1.7 C Standard Library

1.8 C++ and Other C-Based

Languages 1.9 Object Technology

1.10 Typical C Program Development Environment

1.10.1 Phase 1: Creating a Program

1.10.2 Phases 2 and 3: Preprocessing and Compiling a C Program

1.10.3 Phase 4: Linking

1.10.4 Phase 5: Loading

1.10.5 Phase 6: Execution

1.10.6 Problems That May

Occur at Execution Time

1.10.7 Standard Input, Standard Output and Standard Error Streams

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X

1.11.1 Running a C Application from the Windows Command Prompt

1.11.2 Running a C Application Using GNU C with Linux

1.11.3 Running a C Application Using GNU C with Mac OS X

1.12 Operating Systems

1.12.1 Windows—A Proprietary Operating System

1.12.2 Linux—An Open-Source Operating System

1.12.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices

1.12.4 Google's Android

1.13 The Internet and World Wide Web 1.14 Some Key Software Development Terminology

1.15 Keeping Up-to-Date with Information Technologies

1.16 Web Resources

Terminology | Self-Review Exercises | Answers to Self-Review Exercises | Exercises Making a Difference

1.1 Introduction

Welcome to C and C++! C is a concise yet powerful computer programming language that's appropriate for technically oriented people with little or no programming

experience and for experienced programmers to use in building substantial software systems. *C How to Program, Seventh Edition*, is an effective learning tool for each of these audiences.

The core of the book emphasizes effective software engineering through the proven methodologies of *structured programming* in C and *object-oriented programming* in C++. The book presents hundreds of *complete working programs* and shows the outputs produced when those programs are run on a computer. We call this the “live-code approach.” All of these example programs may be downloaded from our website www.deitel.com/books/cht7/.

Most people are familiar with the exciting tasks that computers perform. Using this textbook, you’ll learn how to command computers to perform those tasks. It’s **software** (i.e., the instructions you write to command computers to perform **actions** and make **decisions**) that controls computers (often referred to as **hardware**).

1.2 Computers and the Internet in Industry and Research

These are exciting times in the computer field. Many of the most influential and successful businesses of the last two decades are technology companies, including Apple, IBM, Hew

1.2 Computers and the Internet in Industry and Research 3

lett Packard, Dell, Intel, Motorola, Cisco, Microsoft, Google, Amazon, Facebook, Twitter, Groupon, Foursquare, Yahoo!, eBay and many more. These companies are major employers of people who study computer science, computer engineering, information systems or related disciplines. At the time of this writing, Apple was the most valuable company in the world. Figure 1.1 provides a few examples of the ways in which computers are used in research and industry.

Electronic health records

of error and increases overall efficiency of the health care system.

Human Genome Project

These might include a patient's medical history, prescriptions, immunizations, lab results, allergies, insurance information and more. Making this information available to health care providers across a secure network improves patient care, reduces the probability

The Human Genome Project was founded to identify and analyze the 20,000+ genes in human DNA. The project used computer programs to analyze complex genetic data, determine the sequences of the billions of chemical base pairs that make up human DNA and store the information in databases which have been made available over the Internet to researchers in many fields.

AMBER™ Alert The AMBER (America's Missing: Broadcast Emergency Response)

Alert System is used to find abducted children. Law enforcement notifies TV and radio broadcasters and state transportation officials, who then broadcast alerts on TV, radio, computerized highway signs, the Internet and wireless devices. AMBER Alert recently partnered with Facebook, whose users can “Like” AMBER Alert pages by location to receive alerts in their news feeds.

World

Community Grid

People worldwide can donate their unused computer processing power by installing a free

secure software program that allows the World Community Grid (www.worldcommunitygrid.org) to harness unused capacity. This computing power,

accessed over the Internet, is used in place of world countries, fighting cancer, growing more expensive supercomputers to conduct nutritious rice for regions fighting hunger and scientific research projects that are making a difference—providing clean water to third

Medical imaging X-ray computed tomography (CT) scans, also called CAT (computer ized axial tomography) scans, take X-rays of the body from hundreds of different angles.

Computers are used to adjust the intensity of the X rays, optimizing the scan for each type of tissue, then to combine all of the information to create a 3D image. MRI scanners use a technique called magnetic resonance imaging, also to produce internal images non-invasively.

One Laptop Per Child (OLPC)
One Laptop Per Child (one.laptop.org) is providing low-power, inexpensive,

Internet-enabled laptops to children in third-world countries—enabling learning and reducing the digital divide.

Fig. 1.1 | A few uses for computers. (Part 1 of 3.)

4 Chapter 1 Introduction to Computers, the Internet and the

Web

Cloud computing hardware to ensure that you have enough storage and processing power to meet your needs at their peak levels. Business **Cloud computing** allows you to use software, hardware and information stored in the applications often are expensive, and require “cloud”—i.e., accessed on remote computers significant hardware to run them and via the Internet and available on knowledgeable support staff to ensure that demand—rather than having it stored on your they’re running properly and securely. Using personal computer. These services allow you cloud computing services shifts the burden of to increase or decrease resources to meet your managing these applications from the needs at any given time, so they can be more business to the service provider, saving cost effective than purchasing expensive businesses money.

GPS Global Positioning System (GPS) devices use a network of satellites to retrieve location-based information. Multiple satellites send time stamped signals to the GPS device, which calculates the distance to each satellite based on the time the signal left the satellite and the time the signal arrived. This information is used to determine the exact location of the device. GPS devices can provide step-by-step directions and help you locate nearby businesses (restaurants, gas stations, etc.) and points of interest. GPS is used in numerous location-based Internet services such as check-in apps to help you find your friends (e.g., Four square and Facebook), exercise apps such as RunKeeper that track the time, distance and average speed of your outdoor jog, dating apps that help you find a match nearby and apps that dynamically update changing traffic conditions.

Robots Robots can be used for day-to-day tasks (e.g., iRobot’s Roomba vacuuming robot), entertainment (e.g., robotic pets), military combat, deep sea and space exploration (e.g., NASA’s Mars rover) and more.

RoboEarth (www.robearth.org) is “a World Wide Web for robots.” It allows robots to learn from each other by sharing information and thus improving their abilities to perform tasks, navigate, recognize objects

and more.

E-mail, Instant Messaging,

Video Chat and FTP

Internet-based servers support all of your online messaging. E-mail messages go through a mail server that also stores the messages.

Instant Messaging (IM) and Video Chat apps, such as AIM, Skype, Yahoo! Messenger and

others allow you to communicate with others in real time by sending your messages and live video through servers. FTP (file transfer protocol) allows you to exchange files between multiple computers (e.g., a client computer such as your desktop and a file server) over the Internet.

Internet TV Internet TV set-top boxes (such as Apple TV, Google TV and TiVo) allow you to access an enormous amount of content on demand, such as games, news, movies, television shows and more, and they help ensure that the content is streamed to your TV smoothly.

Fig. 1.1 | A few uses for computers. (Part 2 of 3.)

1.3 Hardware and Software

Game

programming

Analysts expect global video game revenues to reach \$91 billion by 2015

(www.vg247.com/2009/06/23/global-industry-analysis-predicts-gaming-market-to-reach-91-billion-by-2015/).

The most sophisticated games can cost as

much as \$100 million to develop. Activision's *Call of Duty: Black Ops*—one of the best-selling games of all time—earned \$360 million in just one day (www.forbes.com/sites/insertcoin/2011/03/11/call-of-duty-black-ops-now-the-best-selling-video-game-of-all-time/)! Online

social gaming, which enables users worldwide to compete with one another over the Internet, is growing rapidly. Zynga—creator of popular online games such as *Farmville* and *Mafia Wars*—was founded in 2007 and already has over 200 million monthly users. To accommodate the growth in traffic, Zynga is adding nearly 1,000 servers each week (techcrunch.com/2010/09/22/zynga-moves-1-petabyte-of-data-daily-adds-1000-servers-a-week/)!

Fig. 1.1 | A few uses for computers. (Part 3 of 3.)

1.3 Hardware and Software

In use today are more than a billion general-purpose computers, and billions more *embedded* computers are used in cell phones, smartphones, tablet computers, home appliances, automobiles and more. Computers can perform computations and make logical decisions phenomenally faster than human beings can. Many of today's personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime. *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second! In 2011, Fujitsu announced that its “K” supercomputer can perform over 10 quadrillion calculations per second (10 *petaflops*)! To put that in perspective, *the K super computer can perform in one second more than 1,000,000 calculations for every person on the planet!* And—these “upper limits” are growing quickly!

Computers process data under the control of sequences of instructions called **computer programs**. These programs guide the computer through ordered actions specified by people called computer **programmers**. The programs that run on a computer are referred to as **software**. In this book, you'll learn key programming methodologies that are enhancing programmer productivity, thereby reducing software development costs—*structured programming* (in C) and *object-oriented programming* in C++.

A computer consists of various devices referred to as hardware (e.g., the keyboard, screen, mouse, hard disks, memory, DVD drives and processing units). Computing costs are *dropping dramatically*, owing to rapid developments in hardware and software technologies. Computers that might have filled large rooms and cost millions of dollars decades ago are now inscribed on silicon chips smaller than a fingernail, costing perhaps a few dollars each. Ironically, silicon is one of the most abundant materials—it's an ingredient in common sand. Silicon-chip technology has made computing so economical that computers have become a commodity.

6 Chapter 1 Introduction to Computers, the Internet and the Web

1.3.1 Moore's Law

Every year, you probably expect to pay at least a little more for most products and services. The opposite has been the case in the computer and communications fields, especially with regard to the costs of hardware supporting these technologies. For many decades, hardware costs have fallen rapidly. Every year or two, the capacities of computers have approximately *doubled* inexpensively. This remarkable trend often is called **Moore's Law**, named for the person who identified it, Gordon Moore, co-founder of Intel—the leading manufacturer of the processors in today's computers and embedded systems. Moore's Law and related observations apply especially to the amount of memory that computers have for programs, the amount of secondary storage (such as disk storage) they have to hold programs and data over longer periods of time, and their processor speeds—the speeds at which computers execute their programs (i.e., do their work). Similar growth has occurred in the communications field, in which costs have plummeted as enormous demand for communications bandwidth (i.e., information-carrying capacity) has attracted intense competition. We know of no other fields in which technology improves so quickly and costs fall so rapidly. Such phenomenal improvement is truly fostering the *Information Revolution*.

1.3.2 Computer Organization

Regardless of differences in physical appearance, computers can be envisioned as divided into various **logical units** or sections (Fig. 1.2).

Input unit This “receiving” section obtains information (data and computer programs) from **input devices** and places it at the disposal of the other units for processing. Most information is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”), receiving video from a webcam and having your computer receive information from the Internet (such as when you download videos from YouTube™ or e books from Amazon). Newer forms of input include position data from a GPS device, and motion and orientation information from an accelerometer in a smartphone or game controller (such as Microsoft® Kinect™, Wii™ Remote and PlayStation® Move).

Output unit This “shipping” section takes information that the computer has processed and places it on various **output devices** to make it available

for use outside the computer. Most information that's output from computers today is displayed on screens, printed on paper, played as audio or video on PCs and media players (such as Apple's popular iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and "intelligent" appliances.

Fig. 1.2 | Logical units of a computer. (Part 1 of 2.)

1.4 Data Hierarchy 7

Memory unit This rapid-access, relatively low-capacity "warehouse" section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is *volatile*—it's typically lost when the computer's power is turned off. The memory unit is often called either **memory** or **primary memory**. Typical main memories on desktop and notebook computers contain between 1 and 8 GB (GB stands for gigabytes; a gigabyte is approximately one billion bytes).

Arithmetic and logic unit (ALU)

memory unit to certain output devices. Many of today's computers have multiple CPUs and, hence, can perform many operations simultaneously. A **multi-core processor** implements multiple processors on a single integrated-circuit chip—a *dual-core processor* has two CPUs and a *quad-core processor* has four CPUs. Today's desktop computers have processors that can execute billions of instructions per second. This is the long-term, high-capacity "warehousing" section.

Central processing unit (CPU)

Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your *hard drive*) until they're again needed, possibly hours, days, months or even years later. Information on secondary storage devices is *persistent*—it's preserved even when the computer's power is turned off. Secondary storage information takes much longer to access than information in primary memory, but the cost per unit of secondary storage is much less than that of primary memory. Examples of secondary storage devices include CD drives, DVD drives and flash drives, some of which can hold up to 512 GB. Typical hard drives on desktop and notebook computers can hold up to 2 TB (TB stands for terabytes; a terabyte is approximately one trillion bytes).

Secondary storage unit

This "manufacturing" section performs *calculations*, such as addition, subtraction, multiplication and division. It also contains the *decision* mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they're equal. In today's systems, the ALU is usually implemented as part of the next logical unit, the CPU.

This "administrative" section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the

Fig. 1.2 | Logical units of a computer. (Part 2 of 2.)

1.4 Data Hierarchy

Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from bits to characters to fields, and so on.

Figure 1.3

8 Chapter 1 Introduction to Computers, the Internet and the Web

illustrates a portion of the data hierarchy. Figure 1.4 summarizes the data hierarchy’s levels.

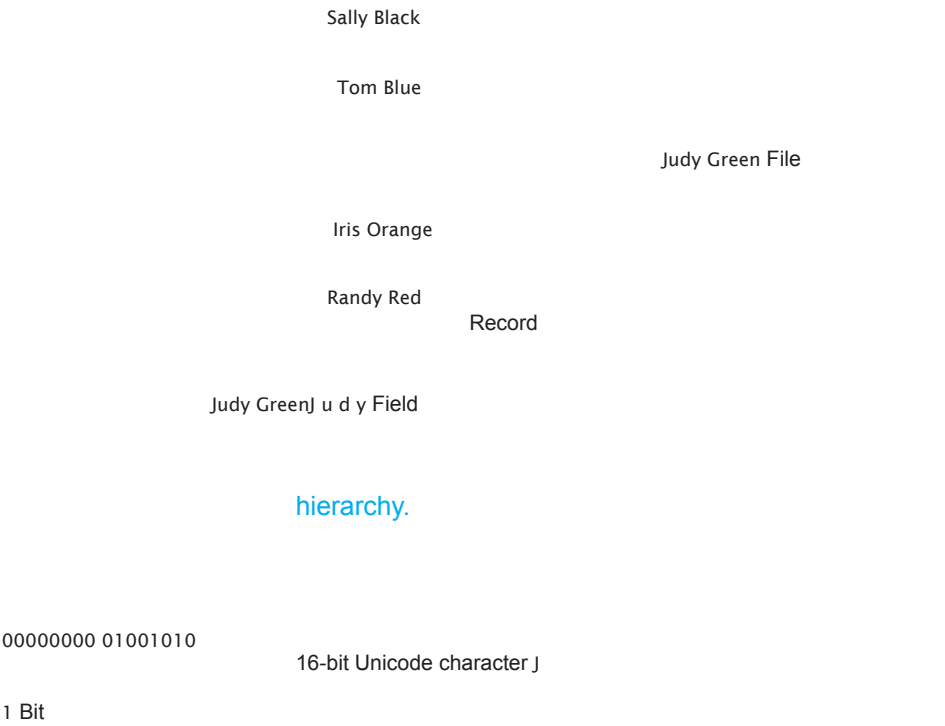


Fig. 1.3 | Data

Bits The smallest data item in a computer can assume the value 0 or the value 1. Such a data item is called a **bit** (short for “binary digit”—a digit that can assume one of two values). It’s remarkable that the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s— *examining a bit’s value, setting a bit’s value and reversing a bit’s value* (from 1 to 0 or from 0 to 1).

Characters It's tedious for people to work with data in the low-level form of bits. Instead, they prefer to work with *decimal digits* (0–9), *letters* (A–Z and a–z), and *special symbols* (e.g., \$, @, %, &, *, (,), –, +, ", :, ? and /). Digits, letters and special symbols are known as **characters**.

The computer's **character set** is the set of all the characters used to write programs and represent data items. Computers process only 1s and 0s, so every character is represented as a pattern of 1s and 0s. The **Unicode** character set contains characters for many of the world's languages. C supports several character sets, including 16-bit Unicode[®] characters

Fig. 1.4 | Levels of the data hierarchy. (Part 1 of 2.)

1.5 Programming Languages 9

	Standard Code for Information Interchange
Characters (cont.) that are composed of two bytes , each composed of eight bits. See Appendix B for more information on the ASCII (American	character set—the popular subset of Unicode that represents uppercase and lowercase letters, digits and some common special characters.
Fields Just as characters are composed of bits, fields are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters could be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.	
Records Several related fields can be used to compose a record . In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses): • Employee identification number (a whole number) • Name (a string of characters) • Address (a string of characters) • Hourly pay rate (a number with a decimal point) • Year-to-date earnings (a number with a decimal point) • Amount of taxes withheld (a number with a decimal point) Thus, a record is a group of related fields. In the preceding example, all the fields belong to the same employee. A company might have many employees and a payroll record for each one.	
Files A file is a group of related records. [Note: More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a <i>sequence of bytes</i> —any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer.] It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.	
Database A database is an electronic collection of data that's organized for easy access and manipulation. The most popular database model is the relational database in which data is stored in simple <i>tables</i> . A table includes <i>records</i> and <i>fields</i> . For example, a table of students might include first name, last name, major, year, student ID number and grade point average. The data for each student is a record, and the individual pieces of information in each record are the fields. You can search, sort and manipulate the data based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with databases of courses, on-campus housing, meal plans, etc.	

1.5 Programming Languages

Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps.

10 Chapter 1 Introduction to Computers, the Internet and the Web

Machine Languages

Any computer can directly understand only its own **machine language**, defined by its hardware architecture. Machine languages generally consist of numbers (ultimately reduced to 1s and 0s). Such languages are cumbersome for humans.

Assembly Languages

Programming in machine language was simply too slow and tedious for most programmers. Instead, they began using Englishlike *abbreviations* to represent elementary operations. These abbreviations formed the basis of **assembly languages**. *Translator programs* called **assemblers** were developed to convert assembly-language programs to machine language. Although assembly-language code is clearer to humans, it's incomprehensible to computers until translated to machine language.

High-Level Languages

To speed the programming process even further, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks. High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical expressions. Translator programs called **compilers** convert high-level language programs into machine language.

The process of compiling a large high-level language program into machine language can take a considerable amount of computer time. **Interpreter** programs were developed to execute high-level language programs directly (without the need for compilation), although more slowly than compiled programs. **Scripting languages** such as JavaScript and PHP are processed by interpreters.

Performance Tip 1.1

Interpreters have an advantage over compilers in Internet scripting. An interpreted program can begin executing as soon as it's downloaded to the client's machine, without needing to be compiled before it can execute. On the downside, interpreted scripts generally run slower than compiled code.

1.6 The C Programming Language

C evolved from two previous languages, BCPL and B. BCPL was developed in 1967 by Martin Richards as a language for writing operating systems and compilers. Ken Thompson modeled many features in his B language after their counterparts in BCPL, and in 1970 he used B to create early versions of the UNIX operating system at Bell Laboratories.

The C language was evolved from B by Dennis Ritchie at Bell Laboratories and was originally implemented in 1972. C initially became widely known as the development language of the UNIX operating system. Many of today's leading operating systems are written in C and/or C++. C is mostly hardware independent—with careful design, it's



possible to write C programs that are **portable** to most computers.

Built for Performance

C is widely used to develop systems that demand performance, such as operating systems, embedded systems, real-time systems and communications systems (Figure 1.5).

1.6 The C Programming Language

11

Operating systems C's portability and performance make it desirable for implementing operating systems, such as Linux and portions of Microsoft's Windows and Google's Android. Apple's OS X is built in Objective-C, which was derived from C. We discuss some key popular desktop/notebook operating systems and mobile operating systems in Section 1.12.

Embedded systems The vast majority of the microprocessors produced each year are embedded in devices other than general purpose computers. These **embedded systems** include navigation systems, smart home appliances, home security systems, smartphones, robots, intelligent traffic intersections and more. C is one of the most popular programming languages for developing embedded systems, which typically need to run as fast as possible and conserve memory. For example, a car's anti-lock brakes must respond immediately to slow or stop the car without skidding; game controllers used for video games should respond instantaneously to prevent any lag between the controller and the action in the game, and to ensure smooth animations.

Real-time systems Real-time systems are often used for "mission-critical" applications that require nearly instantaneous response times. For example, an air-traffic-control system must constantly monitor the positions and velocities of the planes and report that information to air-traffic controllers without delay so that they can alert the planes to change course if there's a possibility of a collision.

Communications systems Communications systems need to route massive amounts of data to their destinations quickly to ensure that things such as audio and video are delivered smoothly and without delay.

Fig. 1.5 | Some popular performance-oriented C applications.

By the late 1970s, C had evolved into what's now referred to as "traditional C." The publication in 1978 of Kernighan and Ritchie's book, *The C Programming Language*, drew wide attention to the language. This became one of the most successful computer

science books of all time.

Standardization

The rapid expansion of C over various types of computers (sometimes called **hardware platforms**) led to many variations that were similar but often incompatible. This was a serious problem for programmers who needed to develop code that would run on several

12 Chapter 1 Introduction to Computers, the Internet and the Web

platforms. It became clear that a standard version of C was needed. In 1983, the X3J11 technical committee was created under the American National Standards Committee on Computers and Information Processing (X3) to “provide an unambiguous and machine independent definition of the language.” In 1989, the standard was approved as ANSI X3.159-1989 in the United States through the **American National Standards Institute (ANSI)**, then worldwide through the **International Standards Organization (ISO)**. We call this simply Standard C. This standard was updated in 1999—its standards document is referred to as *INCITS/ISO/IEC 9899-1999* and often referred to simply as C99. Copies may be ordered from the American National Standards Institute (www.ansi.org) at web.store.ansi.org/ansidocstore.

The New C Standard

We also introduce the new C standard (referred to as C11), which was approved as this book went to publication. The new standard refines and expands the capabilities of C. Not all popular C compilers support the new features. Of those that do, most implement only a subset of the new features. We’ve integrated into the text (and appendices) in easy-to-include-or-omit sections many of the new features implemented in leading compilers.



Portability Tip 1.1

Because C is a hardware-independent, widely available language, applications written in C often can run with little or no modification on a range of different computer systems.

1.7 C Standard Library

As you’ll learn in Chapter 5, C programs consist of pieces called **functions**. You can program all the functions that you need to form a C program, but most C programmers take advantage of the rich collection of existing functions called the **C Standard Library**. Thus, there are really two parts to learning how to program in C—learning the C language itself and learning how to use the functions in the C Standard Library. Throughout the book, we discuss many of these functions. P. J. Plauger’s book *The Standard C Library* is must reading for programmers who need a deep understanding of the library functions, how to implement them and how to use them to write portable code. We use and explain many C library functions throughout this text. Visit the following website for the C Standard Library documentation:

www.dinkumware.com/manuals/#Standard%20C%20Library

C How to Program, 7/e encourages a *building-block approach* to creating programs. Avoid “reinventing the wheel.” Instead, use existing pieces—this is called **software reuse**. When programming in C you’ll typically use the following building blocks:

- C Standard Library functions

- Functions you create yourself
- Functions other people (whom you trust) have created and made available to you

The advantage of creating your own functions is that you'll know exactly how they work. You'll be able to examine the C code. The disadvantage is the time-consuming effort that goes into designing, developing and debugging new functions.

1.8 C++ and Other C-Based Languages 13

Performance Tip 1.2

Using Standard C library functions instead of writing your own comparable versions can improve program performance, because these functions are carefully written to perform efficiently.

Portability Tip 1.2

Using Standard C library functions instead of writing your own comparable versions can improve program portability, because these functions are used in virtually all Standard C implementations.

1.8 C++ and Other C-Based Languages

C++ was developed by Bjarne Stroustrup at Bell Laboratories. It has its roots in C, providing a number of features that “spruce up” the C language. More important, it provides capabilities for **object-oriented programming**. **Objects** are essentially reusable software **components** that model items in the real world. Using a modular, object-oriented design and implementation approach can make software development groups more productive.

Chapters 15–24 present a condensed treatment of C++ selected from our book *C++ How to Program, 8/e*. As you study C++, check out our online C++ Resource Center at www.deitel.com/cplusplus/. Figure 1.6 introduces several other popular C-based programming languages.

Objective-C Objective-C is an object-oriented language based on C. It was developed in the early 1980s and later acquired by NeXT, which in turn was acquired by Apple. It has become the key programming language for the Mac OS X operating system and all iOS-based devices (such as iPods, iPhones and iPads).

Visual C# Microsoft's three primary object-oriented programming languages are Visual Basic, Visual C++ (based on C++) and C# (based on C++ and Java, and developed for integrating the Internet and the web into computer applications).

Java Sun Microsystems in 1991 funded an internal corporate research project which resulted in the C++-based object-oriented programming language called Java. A key goal of Java is to enable the writing of programs that will run on a broad variety of computer systems and computer-controlled devices. This is sometimes called “write once, run anywhere.” Java is used to develop large-scale enterprise applications, to enhance the functionality of web servers (the computers that provide the content we see in our web browsers), to provide applications for consumer devices (smartphones, television set-top boxes and more) and for many other purposes.

PHP PHP—an object-oriented, open-source (see Section 1.12) scripting language based on C and supported by a community of users and developers—is used by many websites including Wikipedia and Facebook. PHP is *platform independent*—implementations exist for all major UNIX, Linux, Mac and Windows operating systems. PHP also supports many databases, including MySQL. Other languages similar in concept to PHP are Perl and Python.

JavaScript JavaScript—developed by Netscape—is the most widely used scripting language. It's primarily used to add programmability to web pages—for example, animations and interactivity with the user. It's provided with all major web browsers.

Fig. 1.6 | Popular C-based programming languages. (Part 2 of 2.)

1.9 Object Technology

Building software quickly, correctly and economically remains an elusive goal at a time when demands for new and more powerful software are soaring. *Objects*, or more precisely the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software developers are discovering that using a modular, object-oriented design-and implementation approach can make software-development groups much more productive than was possible with earlier techniques—object-oriented programs are often easier to understand, correct and modify.

The Automobile as an Object

Let's begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the design of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal hides the mechanisms that slow the car, and the steering wheel *hides* the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make the car go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

Methods and Classes

Let's use our car example to introduce some key object-oriented programming concepts.

ments that actually perform its tasks. It hides these statements from its user, just as a car's accelerator pedal hides from the driver the mechanisms of making the car go faster. In object-oriented programming languages, we create a program unit called a **class** to house the set of methods that perform the class's tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account's current balance is. A class is similar in concept to a car's engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

Instantiation

Just as someone has to *build a car* from its engineering drawings before you can actually drive a car, you must *build an object* from a class before a program can perform the tasks that the class's methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

Reuse

Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems, because existing classes and components often have gone through extensive *testing*, *debugging* and *performance tuning*. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

Messages and Method Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a **method call** that tells a method of the object to perform its task. For example, a program might call a particular bank-account object's *deposit* method to increase the account's balance.

Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading). Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class's **instance variables**.

Encapsulation

Classes **encapsulate** (i.e., wrap) attributes and methods into objects—an object's

attributes and methods are intimately related. Objects may communicate with one another, but not-

16 Chapter 1 Introduction to Computers, the Internet and the Web

mally they're not allowed to know how other objects are implemented—implementation details are *hidden* within the objects themselves. This **information hiding** is crucial to good software engineering.

Inheritance

A new class of objects can be created quickly and conveniently by **inheritance**—the new class absorbs the characteristics of an existing class, possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

1.10 Typical C Program Development Environment

C systems generally consist of several parts: a program development environment, the language and the C Standard Library. The following discussion explains the typical C development environment shown in Fig. 1.7.

C programs typically go through six phases to be executed (Fig. 1.7). These are: **edit**, **preprocess**, **compile**, **link**, **load** and **execute**. Although *C How to Program, Seventh Edition* is a generic C textbook (written independently of the details of any particular operating system), we concentrate in this section on a typical Linux-based C system. [Note: The programs in this book will run with little or no modification on most current C systems, including Microsoft Windows-based systems.] If you're not using a Linux system, refer to the documentation for your system or ask your instructor how to accomplish these tasks in your environment. Check out our C Resource Center at www.deitel.com/C to locate “getting started” tutorials for popular C compilers and development environments.

1.10.1 Phase 1: Creating a Program

Phase 1 consists of editing a file. This is accomplished with an **editor program**. Two editors widely used on Linux systems are **vi** and **emacs**. Software packages for the C/C++ integrated program development environments such as Eclipse and Microsoft Visual Studio have editors that are integrated into the programming environment. You type a C program with the editor, make corrections if necessary, then store the program on a secondary storage device such as a hard disk. C program file names should end with the **.c** extension.

1.10.2 Phases 2 and 3: Preprocessing and Compiling a C Program

In Phase 2, you give the command to **compile** the program. The compiler translates the C program into machine language-code (also referred to as **object code**). In a C system, a **preprocessor** program executes automatically before the compiler's translation phase begins. The **C preprocessor** obeys special commands called **preprocessor directives**, which indicate that certain manipulations are to be performed on the program before compilation. These manipulations usually consist of including other files in the file to be compiled and performing various text replacements. The most common preprocessor directives are discussed in the early chapters; a detailed

discussion of preprocessor features appears in Chapter 13.

In Phase 3, the compiler translates the C program into machine-language code. A **syntax error** occurs when the compiler cannot recognize a statement because it violates the

1.10 Typical C Program Development Environment 17

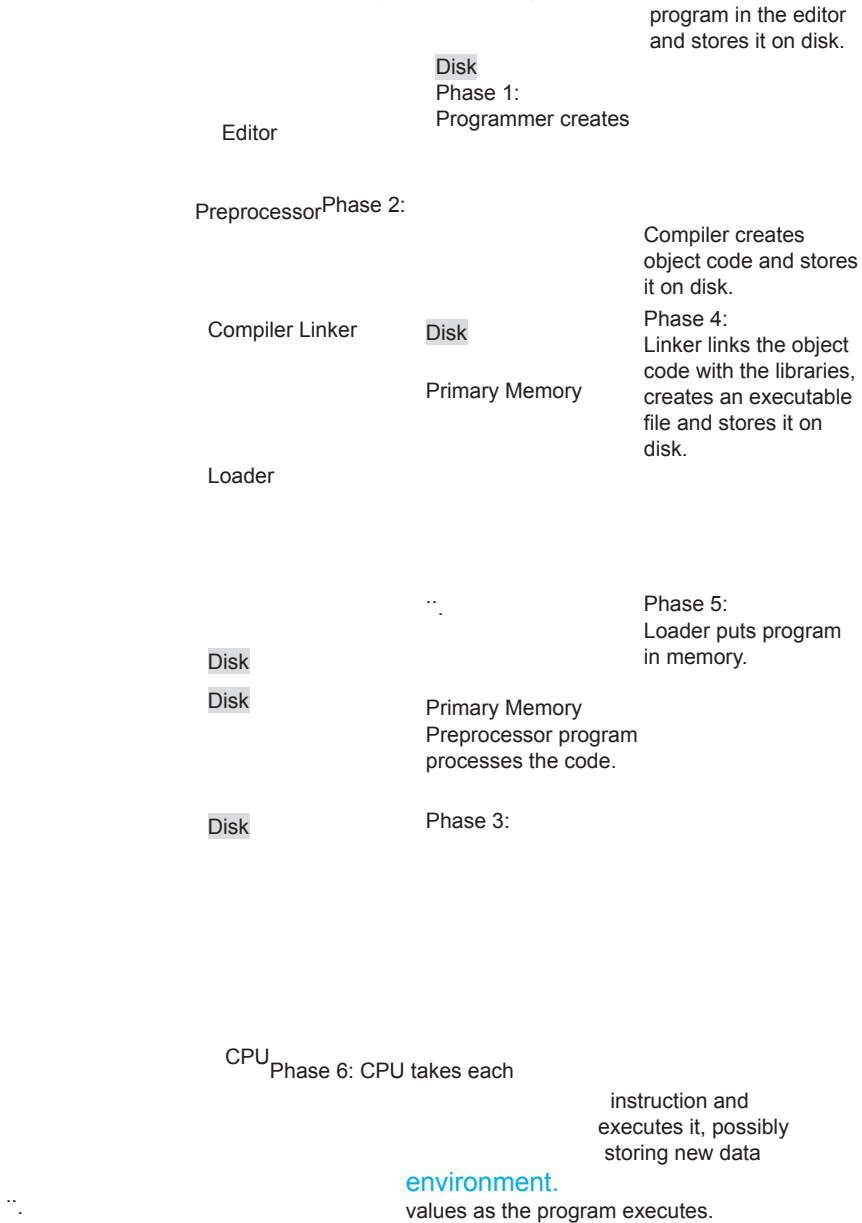


Fig. 1.7 | Typical C development

rules of the language. The compiler issues an error message to help you locate and fix the incorrect statement. The C Standard does not specify the wording for error messages issued by the compiler, so the error messages you see on your system may differ from those on other systems. Syntax errors are also called **compile errors**, or **compile-time errors**.

18 Chapter 1 Introduction to Computers, the Internet and the Web

1.10.3 Phase 4: Linking

The next phase is called **linking**. C programs typically contain references to functions defined elsewhere, such as in the standard libraries or in the private libraries of groups of programmers working on a particular project. The object code produced by the C compiler typically contains “holes” due to these missing parts. A **linker** links the object code with the code for the missing functions to produce an **executable image** (with no missing pieces). On a typical Linux system, the command to compile and link a program is called **gcc** (the GNU C compiler). To compile and link a program named `welcome.c`, type

```
gcc welcome.c
```

at the Linux prompt and press the *Enter* key (or *Return* key). [Note: Linux commands are case sensitive; make sure that each `c` is lowercase and that the letters in the filename are in the appropriate case.] If the program compiles and links correctly, a file called `a.out` is produced. This is the executable image of our `welcome.c` program.

1.10.4 Phase 5: Loading

The next phase is called **loading**. Before a program can be executed, the program must first be placed in memory. This is done by the **loader**, which takes the executable image from disk and transfers it to memory. Additional components from shared libraries that support the program are also loaded.

1.10.5 Phase 6: Execution

Finally, the computer, under the control of its CPU, **executes** the program one instruction at a time. To load and execute the program on a Linux system, type `./a.out` at the Linux prompt and press *Enter*.

1.10.6 Problems That May Occur at Execution Time

Programs do not always work on the first try. Each of the preceding phases can fail because of various errors that we’ll discuss. For example, an executing program might attempt to divide by zero (an illegal operation on computers just as in arithmetic). This would cause the computer to display an error message. You would then return to the edit phase, make the necessary corrections and proceed through the remaining phases again to determine that the corrections work properly.

Common Programming Error 1.1

Errors such as division-by-zero occur as a program runs, so they are called runtime errors or



execution-time errors. Divide-by-zero is generally a fatal error, i.e., one that causes the program to terminate immediately without successfully performing its job. Nonfatal errors allow programs to run to completion, often producing incorrect results.

1.10.7 Standard Input, Standard Output and Standard Error

Streams Most C programs input and/or output data. Certain C functions take their input from `stdin` (the **standard input stream**), which is normally the keyboard, but `stdin` can be connected to another stream. Data is often output to `stdout` (the **standard output stream**), which is normally the computer screen, but `stdout` can be connected to another stream.

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X 19

When we say that a program prints a result, we normally mean that the result is displayed on a screen. Data may be output to devices such as disks and printers. There's also a **standard error stream** referred to as `stderr`. The `stderr` stream (normally connected to the screen) is used for displaying error messages. It's common to route regular output data, i.e., `stdout`, to a device other than the screen while keeping `stderr` assigned to the screen so that the user can be immediately informed of errors.

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X

In this section, you'll run and interact with your first C application. You'll begin by running a guess-the-number game, which randomly picks a number from 1 to 1000 and prompts you to guess it. If your guess is correct, the game ends. If your guess is not correct, the application indicates whether your guess is higher or lower than the correct number. There's no limit on the number of guesses you can make but you should be able to guess any of the numbers in this range correctly in 10 or fewer tries. There's some nice computer science behind this game—in Section 6.8, Searching Arrays, you'll explore the *binary search* technique.

For this test-drive only, we've modified this application from the exercise you'll be asked to create in Chapter 5. Normally this application *randomly* selects the correct answers. The modified application uses the same sequence of correct answers every time you execute the program (though this may vary by compiler), so you can use the same guesses we use in this section and see the same results.

We'll demonstrate running a C application using the Windows **Command Prompt**, a shell on Linux and a **Terminal** window in Mac OS X. The application runs similarly on all three platforms. After you perform the test drive for your platform, you can try the *randomized* version of the game, which we've provided with each test drive's version of the example in a subfolder named `randomized_version`.

Many development environments are available in which you can compile, build and run C applications, such as GNU C, Dev C++, Microsoft Visual C++, CodeLite, Net Beans, Eclipse, Xcode, etc. Consult your instructor for information on your specific development environment. Most C++ development environments can compile both C and C++ programs.

In the following steps, you'll run the application and enter various numbers to guess the correct number. The elements and functionality that you see in this application are typical of those you'll learn to program in this book. We use fonts to distinguish between features you see on the screen (e.g., the **Command Prompt**) and elements that are not directly related to the screen. We emphasize screen features like titles and menus

(e.g., the File menu) in a semibold sans-serif Helvetica font, and to emphasize filenames, text displayed by an application and values you should enter into an application (e.g., `GuessNumber` or `500`) we use a sans-serif Lucida font. As you've noticed, the **defining occurrence** of each key term is set in **bold blue** type. For the Windows version of the test drive in this section, we've modified the background color of the Command Prompt window to make the Command Prompt windows more readable. To modify the Command Prompt colors on your system, open a Command Prompt by selecting **Start > All Programs > Accessories > Command Prompt**, then right click the title bar and select **Properties**. In the "Command Prompt" Properties dialog box that appears, click the **Colors** tab, and select your preferred text and background colors.

20 Chapter 1 Introduction to Computers, the Internet and the Web

1.11.1 Running a C Application from the Windows Command

Prompt 1. *Checking your setup.* It's important to read the Before You Begin section at www.deitel.com/books/cht7/ to make sure that you've copied the book's examples to your hard drive correctly.

2. *Locating the completed application.* Open a Command Prompt window. To change to the directory for the completed `GuessNumber` application, type `cd C:\examples\ch01\GuessNumber\Windows`, then press *Enter* (Fig. 1.8). The command `cd` is used to change directories.

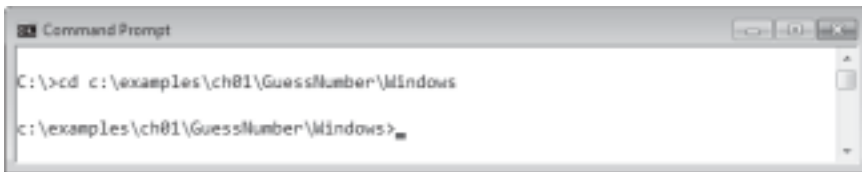


Fig. 1.8 | Opening a Command Prompt window and changing the directory.

3. *Running the `GuessNumber` application.* Now that you are in the directory that contains the `GuessNumber` application, type the command `GuessNumber` (Fig. 1.9) and press *Enter*. [Note: `GuessNumber.exe` is the actual name of the application; however, Windows assumes the `.exe` extension by default.]



Fig. 1.9 | Running the `GuessNumber` application.

4. *Entering your first guess.* The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.9). At the prompt, enter `500` (Fig. 1.10).



```
Command Prompt - GuessNumber
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too high. Try again.
?
```

Fig. 1.10 | Entering your first guess.

5. **Entering another guess.** The application displays "Too high. Try again.", meaning that the value you entered is greater than the number the application chose as

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X 21

the correct guess. So, you should enter a lower number for your next guess. At the prompt, enter 250 (Fig. 1.11). The application again displays "Too high. Try again.", because the value you entered is still greater than the number that the application chose.

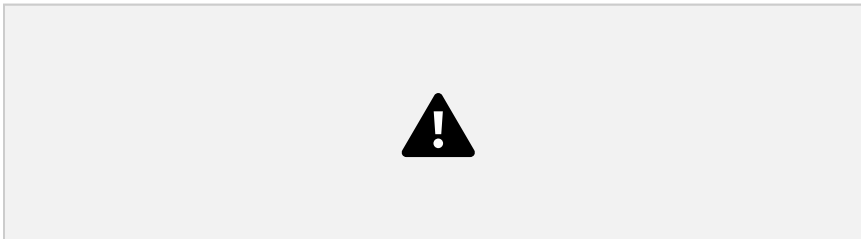


Fig. 1.11 | Entering a second guess and receiving feedback.

6. **Entering additional guesses.** Continue to play the game by entering values until you guess the correct number. The application will display "Excellent! You guessed the number!" (Fig. 1.12).
7. **Playing the game again or exiting the application.** After you guess correctly, the application asks if you'd like to play another game (Fig. 1.12). At the prompt, entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.13), so you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's directory at the Command Prompt

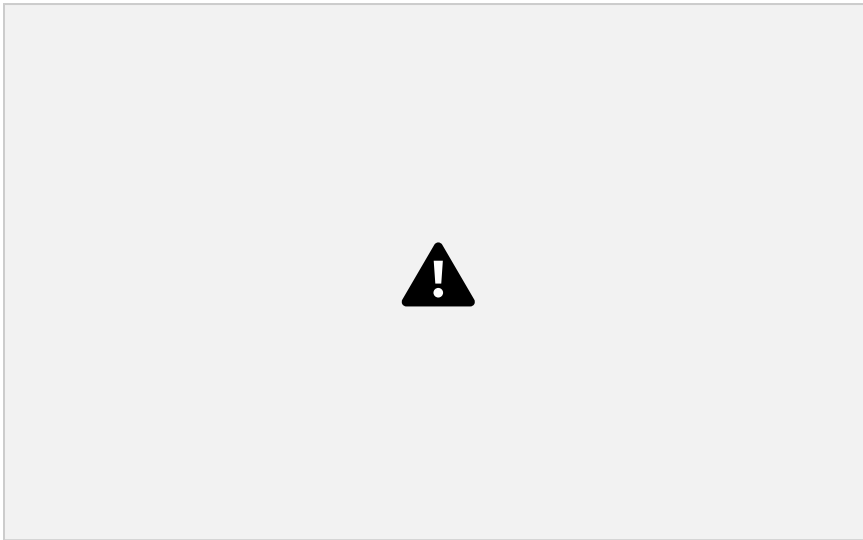


Fig. 1.12 | Entering additional guesses and guessing the correct number.
22 Chapter 1 Introduction to Computers, the Internet and the Web

(Fig. 1.14). Each time you execute this application from the beginning (i.e., *Step 3*), it will choose the same numbers for you to guess.

8. Close the Command Prompt window.

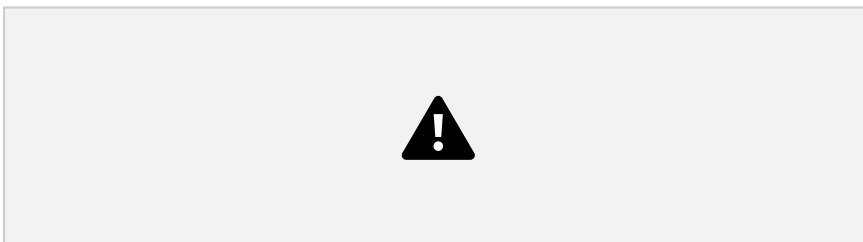


Fig. 1.13 | Playing the game again.



Fig. 1.14 | Exiting the game.

1.11.2 Running a C Application Using GNU C with Linux

For this test drive, we assume that you know how to copy the examples into your home directory. Please see your instructor if you have any questions regarding copying the files to your Linux system. Also, for the figures in this section, we use a bold font to point out the user input required by each step. The prompt in the shell on our system uses the tilde (~) character to represent the home directory, and each prompt ends with the dollar-sign (\$) character. The prompt will vary among Linux systems.

1. **Checking your setup.** It's important to read the Before You Begin section at www.deitel.com/books/cht7/ to make sure that you've copied the book's examples to your hard drive correctly.
2. **Locating the completed application.** From a Linux shell, change to the completed `GuessNumber` application directory (Fig. 1.15) by typing

```
cd examples/ch01/GuessNumber/GNU
```

then pressing *Enter*. The command `cd` is used to change directories.

```
~$ cd examples/ch01/GuessNumber/GNU
~/examples/ch01/GuessNumber/GNU$
```

Fig. 1.15 | Changing to the `GuessNumber` application's directory.

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X 23

3. **Compiling the `GuessNumber` application.** To run an application on the GNU C++ compiler, you must first compile it by typing

```
gcc GuessNumber.c -o GuessNumber
```

as in Fig. 1.16. This command compiles the application and produces an executable file called `GuessNumber`.

```
~/examples/ch01/GuessNumber/GNU$ gcc GuessNumber.c -o GuessNumber
~/examples/ch01/GuessNumber/GNU$
```

Fig. 1.16 | Compiling the `GuessNumber` application using the `gcc` command.

4. **Running the `GuessNumber` application.** To run the executable file `GuessNumber`, type `./GuessNumber` at the next prompt, then press *Enter* (Fig. 1.17).

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

```
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

Fig. 1.17 | Running the `GuessNumber` application.

5. **Entering your first guess.** The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.17). At the prompt, enter 500 (Fig. 1.18).

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

I have a number between 1 and 1000.

Can you guess my number?

Please type your first guess.

? 500

Too high. Try again.

?

Fig. 1.18 | Entering an initial guess.

6. Entering another guess. The application displays "Too high. Try again.", meaning that the value you entered is greater than the number the application chose as the correct guess (Fig. 1.18). At the next prompt, enter 250 (Fig. 1.19). This time the application displays "Too low. Try again.", because the value you entered is less than the correct guess.

7. Entering additional guesses. Continue to play the game (Fig. 1.20) by entering values until you guess the correct number. When you guess correctly, the application displays "Excellent! You guessed the number!"

24 Chapter 1 Introduction to Computers, the Internet and the Web

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

I have a number between 1 and 1000.

Can you guess my number?

Please type your first guess.

? 500

Too high. Try again.

? 250

Too low. Try again.

?

Fig. 1.19 | Entering a second guess and receiving feedback.

Too low. Try again.

? 375

Too low. Try again.

? 437

Too high. Try again.

? 406

Too high. Try again.

? 391

Too high. Try again.

? 383

Too low. Try again.

? 387

Too high. Try again.

? 385

Too high. Try again.

? 384

Excellent! You guessed the number!

Would you like to play again?

Please type (1=yes, 2=no)?

8. **Playing the game again or exiting the application.** After you guess the correct number, the application asks if you'd like to play another game. At the prompt, entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.21) so that you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's directory in the shell (Fig. 1.22). Each time you execute this application from the beginning (i.e., *Step 4*), it will choose the same numbers for you to guess.

```
Excellent! You guessed the number!
Would you like to play again?
Please type ( 1=yes, 2=no )? 1

I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

Fig. 1.21 | Playing the game again.

1.11 Test-Driving a C Application in Windows, Linux and Mac OS X 25

```
Excellent! You guessed the number!
Would you like to play again?
Please type ( 1=yes, 2=no )? 2

~/examples/ch01/GuessNumber/GNU$
```

Fig. 1.22 | Exiting the game.

1.11.3 Running a C Application Using GNU C with Mac OS X For the figures in this section, we use a bold font to point out the user input required by each step. You'll use Mac OS X's **Terminal** window to perform this test drive. To open a **Terminal** window, click the Spotlight Search icon in the upper-right corner of your screen, then type **Terminal** to locate the **Terminal** application. Under **Applications** in the Spotlight Search results, select **Terminal** to open a **Terminal** window. The prompt in a **Terminal** window has the form *hostName:~ userFolder\$* to represent your user directory. For the figures in this section we remove the *hostName:* part and used the generic name *userFolder* to represent your user account's folder.

1. **Checking your setup.** It's important to read the Before You Begin section at www.deitel.com/books/chtp7/ to make sure that you've copied the book's examples to your hard drive correctly. We assume that the examples are located in your user account's **Documents/examples** folder.
2. **Locating the completed application.** In the **Terminal** window, change to the completed **GuessNumber** application directory (Fig. 1.23) by typing

```
cd Documents/examples/ch01/GuessNumber/GNU
```

then pressing *Enter*. The command **cd** is used to change directories.

```
hostName:~ userFolder$ cd Documents/examples/ch01/GuessNumber/GNU
hostName:GNU$
```

Fig. 1.23 | Changing to the GuessNumber application's directory.

3. **Compiling the GuessNumber application.** To run an application on the GNU C compiler, you must first compile it by typing

```
gcc GuessNumber.c -o GuessNumber
```

as in Fig. 1.24. This command compiles the application and produces an executable file called **GuessNumber**.

```
hostName:GNU~ userFolder$ gcc GuessNumber.c -o GuessNumber
hostName:GNU~ userFolder$
```

Fig. 1.24 | Compiling the GuessNumber application using the gcc command.

4. **Running the GuessNumber application.** To run the executable file **GuessNumber**, type `./GuessNumber` at the next prompt, then press *Enter* (Fig. 1.25).

26 **Chapter 1 Introduction to Computers, the Internet and the Web**

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

Fig. 1.25 | Running the GuessNumber application.

5. **Entering your first guess.** The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.25). At the prompt, enter 500 (Fig. 1.26).

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
? 500
Too low. Try again.
?
```

Fig. 1.26 | Entering an initial guess.

6. **Entering another guess.** The application displays "Too low. Try again." (Fig. 1.26), meaning that the value you entered is greater than the number the application chose as the correct guess. At the next prompt, enter 750 (Fig. 1.27). Again the application displays "Too low. Try again.", because the value you entered is less than the correct guess.

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too low. Try again.  
? 750  
Too low. Try again.  
?
```

Fig. 1.27 | Entering a second guess and receiving feedback.

7. **Entering additional guesses.** Continue to play the game (Fig. 1.28) by entering values until you guess the correct number. When you guess correctly, the application displays "Excellent! You guessed the number!"
8. **Playing the game again or exiting the application.** After you guess the correct number, the application asks if you'd like to play another game. At the prompt,

1.12 Operating Systems 27

```
Too low. Try again.  
? 825  
Too high. Try again.  
? 788  
Too low. Try again.  
? 806  
Too low. Try again.  
? 815  
Too high. Try again.  
? 811  
Too high. Try again.  
? 808
```

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )?
```

Fig. 1.28 | Entering additional guesses and guessing the correct number.

entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.29) so you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's folder in the Terminal window (Fig. 1.30). Each time you execute this application from the beginning (i.e., *Step 3*), it will choose the same numbers for you to guess.

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )? 1  
  
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
?
```

Fig. 1.29 | Playing the game again.

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )? 2  
  
hostName:GNU~ userFolder$
```

Fig. 1.30 | Exiting the game.

1.12 Operating Systems

Operating systems are software systems that make using computers more convenient for users, application developers and system administrators. They provide services that allow each application to execute safely, efficiently and *concurrently* (i.e., in parallel) with other

28 Chapter 1 Introduction to Computers, the Internet and the Web

applications. The software that contains the core components of the operating system is called the **kernel**. Popular desktop operating systems include Linux, Windows and Mac OS X. Popular mobile operating systems used in smartphones and tablets include Google's Android, Apple's iOS (for iPhone, iPad and iPod Touch devices), BlackBerry OS and Windows Phone 7. You can develop applications in C for all four of the following key operating systems, including several of the latest mobile operating systems.

1.12.1 Windows—A Proprietary Operating System

In the mid-1980s, Microsoft developed the **Windows operating system**, consisting of a graphical user interface built on top of DOS—an enormously popular personal-computer operating system that users interacted with by *typing* commands. Windows borrowed from many concepts (such as icons, menus and windows) developed by Xerox PARC and popularized by early Apple Macintosh operating systems. Windows 7 is Microsoft's latest operating system—its features include enhancements to the user interface, faster startup times, further refinement of security features, touch-screen and multitouch support, and more. Windows is a *proprietary* operating system—it's controlled by Microsoft exclusively. Windows is by far the world's most widely used operating system.

1.12.2 Linux—An Open-Source Operating System

The Linux operating system is perhaps the greatest success of the *open-source* movement. **Open-source software** departs from the *proprietary* software development style that dominated software's early years. With open-source development, individuals and companies *contribute* their efforts in developing, maintaining and evolving software in exchange for the right to use that software for their own purposes, typically at no charge. Open-source code is often scrutinized by a much larger audience than proprietary software, so errors of ten get removed faster. Open source also encourages

more innovation. Enterprise systems companies, such as IBM, Oracle and many others, have made significant investments in Linux open-source development.

Some key organizations in the open-source community are the Eclipse Foundation (the Eclipse Integrated Development Environment helps programmers conveniently develop software), the Mozilla Foundation (creators of the Firefox web browser), the Apache Software Foundation (creators of the Apache web server used to develop web based applications) and SourceForge (which provides the tools for managing open-source projects—it has over 322,000 of them under development). Rapid improvements to computing and communications, decreasing costs and open-source software have made it much easier and more economical to create a software-based business now than just a decade ago. A great example is Facebook, which was launched from a college dorm room and built with open-source software.

The **Linux** kernel is the core of the most popular open-source, freely distributed, full featured operating system. It's developed by a loosely organized team of volunteers and is popular in servers, personal computers and embedded systems. Unlike that of proprietary operating systems like Microsoft's Windows and Apple's Mac OS X, Linux source code (the program code) is available to the public for examination and modification and is free to download and install. As a result, Linux users benefit from a community of developers

1.12 Operating Systems 29

actively debugging and improving the kernel, an absence of licensing fees and restrictions, and the ability to completely customize the operating system to meet specific needs. A variety of issues—such as Microsoft's market power, the small number of user friendly Linux applications and the diversity of Linux distributions, such as Red Hat Linux, Ubuntu Linux and many others—have prevented widespread Linux use on desktop computers. Linux *has* become extremely popular on servers and in embedded systems, such as Google's Android-based smartphones.

1.12.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices

Apple, founded in 1976 by Steve Jobs and Steve Wozniak, quickly became a leader in personal computing. In 1979, Jobs and several Apple employees visited Xerox PARC (Palo Alto Research Center) to learn about Xerox's desktop computer that featured a graphical user interface (GUI). That GUI served as the inspiration for the Apple Macintosh, launched with much fanfare in a memorable Super Bowl ad in 1984.

The Objective-C programming language, created by Brad Cox and Tom Love at Stepstone in the early 1980s, added capabilities for object-oriented programming (OOP) to the C programming language. Steve Jobs left Apple in 1985 and founded NeXT Inc. In 1988, NeXT licensed Objective-C from StepStone and developed an Objective-C compiler and libraries which were used as the platform for the NeXTSTEP operating system's user interface and Interface Builder—used to construct graphical user interfaces.

Jobs returned to Apple in 1996 when Apple bought NeXT. Apple's Mac OS X operating system is a descendant of NeXTSTEP. Apple's proprietary operating system, **iOS**, is derived from Apple's Mac OS X and is used in the iPhone, iPad and iPod Touch devices.

1.12.4 Google's Android

Android—the fastest growing mobile and smartphone operating system—is based on the Linux kernel and Java. Experienced Java programmers can quickly dive into Android development. One benefit of developing Android apps is the openness of the platform. The operating system is open source and free.

The Android operating system was developed by Android, Inc., which was acquired by Google in 2005. In 2007, the Open Handset Alliance™—a consortium of 34 companies initially and 84 by 2011—was formed to continue developing Android. As of June 2011, more than 500,000 Android smartphones were being activated each day!¹ Android smartphones are now outselling iPhones in the United States.² The Android operating system is used in numerous smartphones (such as the Motorola Droid, HTC EVO™ 4G, Samsung Vibrant™ and many more), e-reader devices (such as the Barnes and Noble Nook™), tablet computers (such as the Dell Streak and the Samsung Galaxy Tab), in-store touch-screen kiosks, cars, robots, multimedia players and more.

1. news.cnet.com/8301-13506_3-20074956-17/google-500000-android-devices-activated-each-day/.

2. www.pcworld.com/article/196035/android_outsells_the_iphone_no_big_surprise.html.

30 Chapter 1 Introduction to Computers, the Internet and the Web

1.13 The Internet and World Wide Web

The Internet—a global network of computers—was made possible by the *convergence of computing and communications technologies*. In the late 1960s, ARPA (the Advanced Research Projects Agency) rolled out blueprints for networking the main computer systems of about a dozen ARPA-funded universities and research institutions. Academic research was about to take a giant leap forward. ARPA proceeded to implement the **ARPANET**, which eventually evolved into today's **Internet**. It rapidly became clear that communicating quickly and easily via electronic mail was the key early benefit of the ARPANET. This is true even today on the Internet, which facilitates communications of all kinds among the world's Internet users.

Packet Switching

A primary goal for ARPANET was to allow *multiple* users to send and receive information simultaneously over the *same* communications paths (e.g., phone lines). The network operated with a technique called **packet switching**, in which digital data was sent in small bundles called **packets**. The packets contained *address, error-control and sequencing* information. The address information allowed packets to be *routed* to their destinations. The sequencing information helped in *reassembling* the packets—which, because of complex routing mechanisms, could actually arrive out of order—into their original order for presentation to the recipient. Packets from different senders were intermixed on the *same* lines to efficiently use the available bandwidth. This packet-switching technique greatly reduced transmission costs, as compared with the cost of *dedicated* communications lines.

The network was designed to operate without centralized control. If a portion of the network failed, the remaining working portions would still route packets from senders to receivers over alternative paths for reliability.

TCP/IP

The protocol (i.e., set of rules) for communicating over the ARPANET became known as **TCP**—the **Transmission Control Protocol**. TCP ensured that messages were properly routed from sender to receiver and that they arrived intact.

As the Internet evolved, organizations worldwide were implementing their own networks. One challenge was to get these different networks to communicate. ARPA accomplished this with the development of **IP**—the **Internet Protocol**, truly creating a network of networks, the current architecture of the Internet. The combined set of protocols is now commonly called **TCP/IP**.

World Wide Web, HTML, HTTP

The **World Wide Web** allows you to execute web-based applications and to locate and view multimedia-based documents on almost any subject over the Internet. The web is a relatively recent creation. In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began to develop a technology for sharing information via hyperlinked text documents. Berners-Lee called his invention the **HyperText Markup Language (HTML)**. He also wrote communication protocols to form the backbone of his new information system, which he called the World Wide Web. In particular, he wrote the **Hypertext Transfer Protocol (HTTP)**—a communications protocol used to send information over the web. The **URL (Uniform Resource Locator)** specifies the address (i.e.,

1.14 Some Key Software Development Terminology 31

location) of the web page displayed in the browser window. Each web page on the Internet is associated with a unique URL. **Hypertext Transfer Protocol Secure (HTTPS)** is the standard for transferring encrypted data on the web.

Mosaic, Netscape, Emergence of Web 2.0

Web use exploded with the availability in 1993 of the Mosaic browser, which featured a user-friendly graphical interface. Marc Andreessen, whose team at the National Center for Supercomputing Applications developed Mosaic, went on to found Netscape, the company that many people credit with igniting the explosive Internet economy of the late 1990s.

In 2003 there was a noticeable shift in how people and businesses were using the web and developing web-based applications. The term **Web 2.0** was coined by Dale Dougherty of O'Reilly Media³ in 2003 to describe this trend. Generally, Web 2.0 companies use the web as a platform to create collaborative, community-based sites (e.g., social networking sites, blogs, wikis).

Companies with Web 2.0 characteristics are Google (web search), YouTube (video sharing), Facebook (social networking), Twitter (microblogging), Groupon (social commerce), Foursquare (mobile check-in), Salesforce (business software offered as online services “in the cloud”), Craigslist (mostly free classified listings), Flickr (photo sharing), Skype (Internet telephony and video calling and conferencing) and Wikipedia (a free online encyclopedia).

Web 2.0 *involves* the users—not only do they create content, but they help organize it, share it, remix it, critique it, update it, etc. Web 2.0 is a *conversation*, with everyone having the opportunity to speak and share views. Companies that understand Web 2.0 realize that their products and services are conversations as well.

Architecture of Participation

Web 2.0 embraces an **architecture of participation**—a design that encourages user interaction and community contributions. You, the user, are the most important aspect of Web 2.0—so important, in fact, that in 2006, *TIME* magazine’s “Person of the Year” was “You.”⁴ The article recognized the social phenomenon of Web 2.0—the shift away from a *powerful few* to an *empowered many*. Popular blogs now compete with traditional media powerhouses, and many Web 2.0 companies are built almost entirely on user-generated content. For websites like Facebook®, Twitter™, YouTube, eBay® and Wikipedia® users create the content, while the companies provide the *platforms* on which to enter, manipulate and share the information.

1.14 Some Key Software Development Terminology

Figure 1.31 lists a number of buzzwords that you’ll hear in the software development community. We’ve created Resource Centers on most of these topics, with more on the way.

3. T. O’Reilly, “What is Web 2.0: Design Patterns and Business Models for the Next Generation of Software.” September 2005 <<http://www.oreillynet.com/pub/a/oreilly/tim/news/2005/09/30/what-is-web-2.0.html?page=1>>.

4. L. Grossman, “TIME’s Person of the Year: You.” *TIME*, December 2006 <<http://www.time.com/time/magazine/article/0,9171,1569514,00.html>>.

32 Chapter 1 Introduction to Computers, the Internet and the Web

Ajax **Ajax** is one of the premier Web 2.0 software technologies. Ajax helps Internet-based applications perform like desktop applications—a difficult task, given that such applications suffer transmission delays as data is shuttled back and forth between your computer and servers on the Internet.

Agile software development than previous methodologies. Check out the **Agile software development** is a set of Agile Alliance (www.agilealliance.org) and the methodologies that try to get software Agile Manifesto (www.agilemanifesto.org) implemented faster and using fewer resources

Refactoring **Refactoring** involves reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality. It’s widely employed with agile development methodologies. Many IDEs include *refactoring tools* to do major portions of the reworking automatically.

Design patterns patterns tries to enumerate those recurring **Design patterns** are proven architectures for patterns, encouraging software designers to constructing flexible and maintainable *reuse* them to develop better-quality software object-oriented software. The field of design using less time, money and effort.

LAMP MySQL is an open-source database management system. PHP is the most popular open-source server-side Internet “scripting” language for developing Internet-based applications. **LAMP** is an acronym for the set of open source technologies that many developers use to build web applications—it stands for Linux, Apache, MySQL and PHP (or Perl or Python—two other languages used for similar purposes).

Software as a Service (SaaS)

Platform as a Service (PaaS)

Software

Development Kit (SDK)

Software has generally been viewed as a product; most software still is offered this way. If you want to run an application, you buy a software package from a software vendor—often a CD, DVD or web download. You then install that software on your computer and run it as needed. As new versions of the software appear, you upgrade your software, often requiring significant time

and at considerable expense. This process can become cumbersome for organizations with tens of thousands of systems that must be maintained on a diverse array of computer equipment. With **Software as a Service (SaaS)**, the software runs on servers elsewhere on the Internet. When that server is updated, all clients worldwide see the new capabilities—no local installation is needed. You access the service through a browser. Browsers are quite portable, so you can run the same applications on a wide variety of computers from anywhere in the world. Salesforce.com, Google, and Microsoft's Office Live and Windows Live all offer SaaS. SaaS is a capability of cloud computing.

Platform as a Service (PaaS), another capability of cloud computing, provides a computing platform for developing and running applications as a service over the web, rather than installing the tools on your computer. PaaS providers include Google App Engine, Amazon EC2, Bungee Labs and more. **Software Development Kits (SDKs)** include the tools and documentation developers use to program applications.

Fig. 1.31 | Software technologies.

1.15 Keeping Up-to-Date with Information Technologies

33 Figure 1.32 describes software product-release categories.

Alpha An *alpha* version is the earliest release of a software product that's still under active development. Alpha versions are often buggy, incomplete and unstable and are released to a relatively small number of developers for testing new features, getting early feedback, etc.

Beta *Beta* versions are released to a larger number of developers later in the development process after most major bugs have been fixed and new features are nearly complete. Beta software is more stable, but still subject to change.

Release candidates for a variety of purposes. Any bugs that appear are corrected, and eventually the final product is released to the general public. Software companies often distribute incremental updates over the Internet.

Continuous beta Software that's developed using this approach generally does not have version numbers (for example, Google search or Gmail). The *Release candidates* are generally *feature complete* and (supposedly) bug free and ready for use by the community, which provides a diverse testing environment—the software is used on different systems, with varying constraints and version.

1.15 Keeping Up-to-Date with Information Technologies

Figure 1.33 lists key technical and business publications that will help you stay up-to-date with the latest news and trends and technology. You can also find a growing list of Internet- and web-related Resource Centers at www.deitel.com/resourcecenters.html.

ACM TechNews technews.acm.org/
ACM Transactions on www.gccis.rit.edu/taccess/index.html
Accessible Computing toit.acm.org/
ACM Transactions on Internet
Technology
Bloomberg BusinessWeek www.businessweek.com CNET
news.cnet.com Communications of the ACM cacm.acm.org/
Computer World www.computerworld.com Engadget
www.engadget.com eWeek www.eweek.com

Fig. 1.33 | Technical and business publications. (Part 1 of 2.)

34 Chapter 1 Introduction to Computers, the Internet and the Web

Fast Company www.fastcompany.com/
Fortune money.cnn.com/magazines/fortune/
IEEE Computer www.computer.org/portal/web/computer IEEE Internet
Computing www.computer.org/portal/web/internet/home InfoWorld
www.infoworld.com
Mashable mashable.com
PCWorld www.pcworld.com
SD Times www.sdtimes.com
Slashdot slashdot.org/
Smarter Technology www.smartertechnology.com
Technology Review technologyreview.com
Techcrunch techcrunch.com
Wired www.wired.com

Fig. 1.33 | Technical and business publications. (Part 2 of 2.)

1.16 Web Resources

This section provides links to our C and related Resource Centers that will be useful to you as you learn C. These Resource Centers include various C resources, including blogs, articles, whitepapers, compilers, development tools, downloads, FAQs, tutorials, webcasts, wikis and links to resources for C game programming with the Allegro

libraries. For up dates on Deitel publications, Resource Centers, training courses, partner offers and more, follow us on Facebook[®] at www.facebook.com/deitelfan/ and on Twitter[®] @deitel.

Deitel & Associates Websites

www.deitel.com/books/chtp7/

The Deitel & Associates *C How to Program*, 7/e site. Here you'll find links to the book's examples and other resources.

www.deitel.com/C/

www.deitel.com/visualcplusplus/

www.deitel.com/codeSearchEngines/

www.deitel.com/programmingProjects/

Check these Resource Centers for compilers, code downloads, tutorials, documentation, books, e books, articles, blogs, RSS feeds and more that will help you develop C applications.

www.deitel.com

Check this site for updates, corrections and additional resources for all Deitel publications.

www.deitel.com/newsletter/subscribe.html

Subscribe here for the *Deitel[®] Buzz[™] Online* e-mail newsletter to follow the Deitel & Associates publishing program, including updates and errata to *C How to Program*, 7/e.

Terminology 35

Terminology

actions (computers perform)

2 agile software development

32 Ajax 32

American National Standards Institute
(ANSI) 12

Android 29

architecture of participation 31

arithmetic and logic unit (ALU) 7

ASCII 9

assembler 10

assembly language 10

bit 8

bytes 8

C preprocessor 16

C Standard Library 12

central processing unit (CPU) 7

character 8

character set 8

class 15

cloud computing 4

compile 16

compile error 17

compile phase 16

compile-time error 17

compiler 10

components (software) 13

computer program 5

data hierarchy 7

database 9

decisions (made by computers)

2 design pattern 32

edit phase 16

editor program 16

embedded systems 11

encapsulate 15

executable image 18

execute 18

execute phase 16

field 9

file 9

function 12

gcc compilation command 18

hardware 2

hardware platform 11

high-level language 10

information hiding 16

inheritance 16

input device 6

input unit 6

instance 15

instance variable 15

International Standards Organization (ISO) 12

interpreter 10

iOS 29

kernel 28

LAMP 32

link phase 16

linker 18

linking 18

Linux 28

load phase 16

loader 18

loading 18	preprocessor 16
logical unit 6	preprocessor directive 16
machine language 10	primary memory 7
memory 7	programmer 5
memory unit 7	record 9
method 14	refactoring 32
method call 15	scripting language 10
Moore's Law 6	secondary storage unit 7
multi-core processor 7	software 2
object 13	Software as a Service (SaaS) 32
object code 16	Software Development Kit (SDK)
object-oriented programming (OOP) 13	32 standard error stream (<code>stderr</code>) 19
open source 28	standard input stream (<code>stdin</code>) 18
operating system 27	standard output stream (<code>stdout</code>) 18
output device 6	syntax error 16
output unit 6	Unicode 8
Platform as a Service (PaaS) 32	Web 2.0 31
portable program 10	Windows Operating System 28
preprocess phase 16	

36 Chapter 1 Introduction to Computers, the Internet and the Web

Self-Review Exercises

1.1 Fill in the blanks in each of the following:

- Computers process data under the control of sequences of instructions called `computer`.
- is a type of computer language that uses Englishlike abbreviations for machine-language instructions.
- languages are most convenient to the programmer for writing programs quickly and easily.
- The only language a computer can directly understand is that computer's `.`
- The programs that translate high-level language programs into machine language are called `.`
- With development, individuals and companies contribute their efforts in developing, maintaining and evolving software in exchange for the right to use that software for their own purposes, typically at no charge.
- C is widely known as the development language of the operating system.

1.2 Fill in the blanks in each of the following sentences about the C environment.

- C programs are normally typed into a computer using a(n) program.
- In a C system, a(n) program automatically executes before the translation phase begins.

- The two most common kinds of preprocessor directives are `and` `.`
- The program combines the output of the compiler with various library functions to produce an executable image.
- The program transfers the executable image from disk to memory.

1.3 Fill in the blanks in each of the following statements (based on Section 1.9):

- Objects have the property of `==` although objects may know how to communicate with one another across well-defined interfaces, they normally are not allowed to know how other objects are implemented.

- In object-oriented programming languages, we create to house the set of methods that perform tasks.
- With `,` new classes of objects are derived by absorbing characteristics of existing classes, then adding unique characteristics of their own.
- The size, shape, color and weight of an object are considered of the object's class.

Answers to Self-Review Exercises

1.1 a) programs. b) Assembly language. c) High-level. d) machine language. e) compilers. f) open-source. g) UNIX.

1.2 a) editor. b) preprocessor. c) including other files in the file to be compiled, performing various text replacements. d) linker. e) loader.

1.3 a) information hiding. b) classes. c) inheritance. d) attributes.

Exercises

1.4 Categorize each of the following items as either hardware or software: a)

- CPU
- b) C++ compiler
- c) ALU
- d) C++ preprocessor
- e) input unit
- f) an editor program

Making a Difference 37

1.5 Fill in the blanks in each of the following statements:

- a) The process of instructing the computer to solve a problem is called . b) What type of computer language uses Englishlike abbreviations for machine-language instructions? .
- c) The level of computer language at which it's most convenient to write programs quickly and easily is .
- d) The only language that a computer directly understands is called that computer's .
- e) Web 2.0 embraces a(n) $\Rightarrow$ a design that encourages user interaction and community contributions.

1.6 Fill in the blanks in each of the following statements:

- a) is now used to develop large-scale enterprise applications, to enhance the functionality of web servers, to provide applications for consumer devices and for many other purposes.
- b) initially became widely known as the development language of the UNIX operating system.
- c) The programming language was developed by Bjarne Stroustrup in the early 1980s at Bell Laboratories.

1.7 Discuss the meaning of each of the following names:

- a) `stdin`
- b) `stdout`
- c) `stderr`

1.8 Why is so much attention today focused on object-oriented programming? **1.9 (Internet in Industry and Research)** Figure 1.1 provides examples of how computers and the Internet are being used in industry and research. Find three additional examples and describe how each is using the Internet and the web.

1.10 (Cloud Computing) Describe three benefits of the cloud computing model. **1.11 (Internet Negatives)** Besides their numerous benefits, the Internet and the web have several downsides, such as privacy issues, identity theft, spam and malware. Research some of the negative aspects of the Internet. List five problems and describe what could possibly be done to help solve each.

1.12 (Watch as an Object) You are probably wearing on your wrist one of the most common types of objects—a watch. Discuss how each of the following terms and concepts applies to the notion of a watch: object, attributes, behaviors, class, inheritance (consider, for example, an alarm clock), messages, encapsulation and information hiding.

Making a Difference

1.13 (Test-Drive: Carbon Footprint Calculator) Some scientists believe that carbon emissions, especially from the burning of fossil fuels, contribute significantly to global warming and that this can be combatted if individuals take steps to limit their use of carbon-based fuels. Organizations and individuals are increasingly concerned about their “carbon footprints.” Websites such as

and Carbon Footprint

provide carbon footprint calculators. Test-drive these calculators to estimate your carbon footprint. Exercises in later chapters will ask you to program your own carbon footprint calculator. To pre pare for this, use the web to research the formulas for calculating carbon footprints.

38 Chapter 1 Introduction to Computers, the Internet and the Web

1.14 (Test-Drive: Body Mass Index Calculator) By recent estimates, two-thirds of the people in the United States are overweight and about half of those are obese. This causes significant increases in illnesses such as diabetes and heart disease. To determine whether a person is overweight or obese, you can use a measure called the body mass index (BMI). The United States Department of Health and Human Services provides a BMI calculator at www.nhlbisupport.com/bmi/. Use it to calculate your own BMI. An exercise in Chapter 2 will ask you to program your own BMI calculator. To pre pare for this, use the web to research the formulas for calculating BMI.

1.15 (Gender Neutrality) Many people want to eliminate sexism in all forms of communication. You've been asked to create a program that can process a paragraph of text and replace gender-specific words with gender-neutral ones. Assuming that you've been given a list of gender-specific words and their gender-neutral replacements (e.g., replace "wife" with "spouse," "man" with "per son," "daughter" with "child" and so on), explain the procedure you'd use to read through a para graph of text and manually perform these replacements. How might your procedure generate a strange term like "woperchild?" In Chapter 4, you'll learn that a more formal term for "procedure" is "algorithm," and that an algorithm specifies the steps to be performed and the order in which to perform them.

1.16 (Privacy) Some online e-mail services save all e-mail correspondence for some period of time. Suppose a disgruntled employee were to post all of the e-mail correspondences for millions of people, including yours, on the Internet. Discuss the issues.

1.17 (Programmer Responsibility and Liability) As a programmer in industry, you may develop software that could affect people's health or even their lives. Suppose a software bug in one of your programs causes a cancer patient to receive an excessive dose during radiation therapy and that the person is severely injured or dies. Discuss the issues.

1.18 (2010 "Flash Crash") An example of the consequences of our excessive dependence on computers was the so-called "flash crash" which occurred on May 6, 2010, when the U.S. stock market fell precipitously in a matter of minutes, wiping out trillions of dollars of investments, and then recovered within minutes. Research online the causes of this crash and discuss the issues it raises.

1.19 (Making a Difference Projects) The following is a list of just a few worldwide organizations that are working to make a difference. Visit these sites and our Making a Difference Resource Center at www.deitel.com/makingadifference. Prepare a top-10 list of programming projects that you think could indeed "make a difference."

- www.imaginecup.com/

The *Microsoft Image Cup* is a global competition in which students use technology to try to solve some of the world's most difficult problems, such as environmental sustainability, ending hunger, emergency response, literacy and combating HIV/AIDS. Visit www.imaginecup.com/about for more information about the competition and to learn about the projects developed by previous winners. You can also find several project ideas submitted by worldwide charitable organizations at www.imaginecup.com/students/imagine-cup-solve-this. For additional ideas for programming projects that can make a difference, search the web for "making a difference" and visit the following websites:

- www.un.org/millenniumgoals

The United Nations Millennium Project seeks solutions to major worldwide issues such as environmental sustainability, gender equality, child and maternal health, universal education and more.

- www.ibm.com/smarterplanet/

The IBM® Smarter Planet website discusses how IBM is using technology to solve issues related to business, cloud computing, education, sustainability and more.

Making a Difference 39

- www.gatesfoundation.org/Pages/home.aspx

The Bill and Melinda Gates Foundation provides grants to organizations that work to alleviate hunger, poverty and disease in developing countries. In the United States, the foundation focuses on improving public education, particularly for people with few resources. • www.nethope.org/

NetHope is a collaboration of humanitarian organizations worldwide working to solve technology problems such as connectivity, emergency response and more.

- www.rainforestfoundation.org/home

The Rainforest Foundation works to preserve rainforests and to protect the rights of the indigenous people who call the rainforests home. The site includes a list of things you can do to help. • www.undp.org/

The United Nations Development Programme (UNDP) seeks solutions to global challenges such as crisis prevention and recovery, energy and the environment and democratic governance. • www.unido.org

The United Nations Industrial Development Organization (UNIDO) seeks to reduce poverty, give developing countries the opportunity to participate in global trade, and promote energy efficiency and sustainability.

- www.usaid.gov/

USAID promotes global democracy, health, economic growth, conflict prevention, humanitarian aid and more.

- www.toyota.com/ideas-for-good/

Toyota's Ideas for Good website describes several Toyota technologies that are making a difference—including their Advanced Parking Guidance System, Hybrid Synergy Drive®, Solar Powered Ventilation System, T.H.U.M.S. (Total Human Model for Safety) and Touch Tracer Display. You can participate in the Ideas for Good challenge by submitting a short essay or video describing how these technologies can be used for other good purposes.

Introduction to C

Programming

*What's in a name?
That which we call a rose
By any other name would
smell as sweet.*
—William Shakespeare

“Take some more tea,” the March Hare said to Alice, very earnestly. “I’ve had nothing yet,” Alice replied in an offended tone: “so I can’t take more.” “You mean you can’t take less,” said the Hatter: “it’s very easy to take more than nothing.”

—Lewis Carroll

High thoughts must have high language.

—Aristophanes

Objectives

In this chapter, you’ll:

- Write simple computer programs in C.
- Use simple input and output statements.
- Use the fundamental data types.
- Learn computer memory concepts.
- Use arithmetic operators.
- Learn the precedence of arithmetic operators.
- Write simple decision making statements.

2.1 Introduction

2.2 A Simple C Program: Printing a Line of Text

2.3 Another Simple C Program: Adding Two Integers

2.4 Memory Concepts

2.5 Arithmetic in C

2.6 Decision Making: Equality and Relational Operators

2.7 Secure C Programming

2.1 Introduction 41

Summary | Terminology | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

2.1 Introduction

The C language facilitates a structured and disciplined approach to computer-program design. In this chapter we introduce C programming and present several examples that illustrate many important features of C. Each example is analyzed one statement at a time. In Chapters 3 and 4 we present an introduction to structured programming in C. We then use the structured approach throughout the remainder of the C portion of the text.

2.2 A Simple C Program: Printing a Line of Text

C uses some notations that may appear strange to people who have not programmed computers. We begin by considering a simple C program. Our first example prints a line of text. The program and its screen output are shown in Fig. 2.1.

```
1 // Fig. 2.1: fig02_01.c
2 // A first program in C.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     printf( "Welcome to C!\n" );
9 } // end function main
```

Welcome to C!

Fig. 2.1 | A first program in C.

Comments

Even though this program is simple, it illustrates several important features of the C language. Lines 1 and 2

```
// Fig. 2.1: fig02_01.c
// A first program in C
```

begin with `//`, indicating that these two lines are **comments**. You insert comments to **document programs** and improve program readability. Comments do *not* cause the computer to perform any action when the program is run. Comments are *ignored* by the C compiler and do *not* cause any machine-language object code to be generated. The preceding com

42 Chapter 2 Introduction to C Programming

ment simply describes the figure number, file name and purpose of the program. Comments also help other people read and understand your program.

You can also use `/*...*/` **multi-line comments** in which everything from `/*` on the first line to `*/` at the end of the last line is a comment. We prefer `//` comments because they're shorter and they eliminate common programming errors that occur with `/*...*/` comments, especially when the closing `*/` is omitted.

#include Preprocessor Directive

Line 3

```
#include <stdio.h>
```

is a directive to the **C preprocessor**. Lines beginning with `#` are processed by the preprocessor *before* compilation. Line 3 tells the preprocessor to include the contents of the **standard input/output header** (`<stdio.h>`) in the program. This header contains information used by the compiler when compiling calls to standard input/output library functions such as `printf` (line 8). We explain the contents of headers in more detail in Chapter 5.

Blank Lines and White Space

Line 4 is simply a blank line. You use blank lines, space characters and tab characters (i.e., “tabs”) to make programs easier to read. Together, these characters are known as **white space**. White-space characters are normally ignored by the compiler.

The main Function

Line 6

```
int main( void )
```

is a part of every C program. The parentheses after **main** indicate that **main** is a program building block called a **function**. C programs contain one or more functions, one of which *must* be **main**. Every program in C begins executing at the function **main**. Functions can *return* information. The keyword **int** to the left of **main** indicates that **main** “returns” an integer (whole-number) value. We’ll explain what it means for a function to “return a value” when we demonstrate how to create your own functions in Chapter 5. For now, simply include the keyword **int** to the left of **main** in each of your programs. Functions also can *receive* information when they’re called upon to execute. The **void** in parentheses here means that **main** does *not* receive any information. In Chapter 14, we’ll show an example of **main** receiving information.

Good Programming Practice 2.1

Every function should be preceded by a comment describing the purpose of the function.

A left brace, {, begins the **body** of every function (line 7). A corresponding **right brace** ends each function (line 9). This pair of braces and the portion of the program between the braces is called a *block*. The block is an important program unit in C.

An Output Statement

Line 8

```
printf( "Welcome to C!\n" );
```

2.2 A Simple C Program: Printing a Line of Text 43

instructs the computer to perform an **action**, namely to print on the screen the **string** of characters marked by the quotation marks. A string is sometimes called a **character string**, a **message** or a **literal**. The entire line, including the **printf** function (the “f” stands for “formatted”), its **argument** within the parentheses and the semicolon (;), is called a **statement**. Every statement must end with a semicolon (also known as the **statement terminator**). When the preceding **printf** statement is executed, it prints the message **Welcome to C!** on the screen. The characters normally print exactly as they appear between the double quotes in the **printf** statement.

Escape Sequences

Notice that the characters **\n** were not printed on the screen. The backslash (\) is called an **escape character**. It indicates that **printf** is supposed to do something out of the ordinary. When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an **escape sequence**. The escape sequence **\n** means **newline**. When a newline appears in the string output by a **printf**, the newline causes the cursor to position to the beginning of the next line on the screen. Some common escape sequences are listed in Fig. 2.2.

`\n` Newline. Position the cursor at the beginning of the next line. `\t` Horizontal tab. Move the cursor to the next tab stop. `\a` Alert. Produces a sound or visible alert without changing the current cursor position.

`\\` Backslash. Insert a backslash character in a string.

`\"` Double quote. Insert a double-quote character in a string. **Fig. 2.2 | Some**

common escape sequences .

Because the backslash has special meaning in a string, i.e., the compiler recognizes it as an escape character, we use a double backslash (`\\`) to place a single backslash in a string. Printing a double quote also presents a problem because double quotes mark the boundaries of a string—such quotes are not printed. By using the escape sequence `\"` in a string to be output by `printf`, we indicate that `printf` should display a double quote. The right brace, `}`, (line 9) indicates that the end of `main` has been reached.

Good Programming Practice 2.2

Add a comment to the line containing the right brace, `}`, that closes every function, including `main`.

We said that `printf` causes the computer to perform an **action**. As any program executes, it performs a variety of actions and makes **decisions**. Section 2.6 discusses decision making. Chapter 3 discusses this **action/decision model** of programming in depth.

The Linker and Executables

Standard library functions like `printf` and `scanf` are *not* part of the C programming language. For example, the compiler *cannot* find a spelling error in `printf` or `scanf`. When

44 **Chapter 2 Introduction to C Programming**

the compiler compiles a `printf` statement, it merely provides space in the object program for a “call” to the library function. But the compiler does *not* know where the library functions are—the *linker* does. When the linker runs, it locates the library functions and inserts the proper calls to these library functions in the object program. Now the object program is complete and ready to be executed. For this reason, the linked program is called an **executable**. If the function name is misspelled, the *linker* will spot the error, because it will not be able to match the name in the C program with the name of any known function in the libraries.

Common Programming Error 2.1

Mistyping the name of the output function `printf` as `print` in a program.

Good Programming Practice 2.3

Indent the entire body of each function one level of indentation (we recommend three spaces) within the braces that define the body of the function. This indentation emphasizes the functional structure of programs and helps make programs easier to read.

Good Programming Practice 2.4

Set a convention for the size of indent you prefer and then uniformly apply that convention. The tab key may be used to create indents, but tab stops may vary.

Using Multiple `printf`s

The `printf` function can print **Welcome to C!** several different ways. For example, the program of Fig. 2.3 produces the same output as the program of Fig. 2.1. This works because each `printf` resumes printing where the previous `printf` stopped printing. The first `printf` (line 8) prints **Welcome** followed by a space, and the second `printf` (line 9) begins printing on the *same* line immediately following the space.

```
1 // Fig. 2.3: fig02_03.c
2 // Printing on one line with two printf statements.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     9      _);_printf( "to
printf( "Welcome C!\n" );
10 } // end function main
```

Welcome to C!

Fig. 2.3 | Printing on one line with two `printf` statements.

One `printf` can print *several* lines by using additional newline characters as in Fig. 2.4. Each time the `\n` (newline) escape sequence is encountered, output continues at the beginning of the next line.

2.3 Another Simple C Program: Adding Two Integers 45

```
1 // Fig. 2.4: fig02_04.c
2 // Printing multiple lines with a single printf.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     9      \n \n
printf( "Welcome to C!\n" );
9 } // end function main
```

Welcome
to
C!

Fig. 2.4 | Printing multiple lines with a single `printf`.

2.3 Another Simple C Program: Adding Two Integers

Our next program uses the Standard Library function `scanf` to obtain two integers typed by a user at the keyboard, computes the sum of these values and prints the result using `printf`. The program and sample output are shown in Fig. 2.5. [In the input/output dialog of Fig. 2.5, we emphasize the numbers entered by the user in **bold**.]

```

1 // Fig. 2.5: fig02_05.c
2 // Addition program.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     9 10 11                second number to be entered by
    int integer1; // first number to be user int sum; // variable in which
    entered by user int integer2; // sum will be stored

12 printf( "Enter first integer\n" ); // prompt
13     14                an integer
    scanf( "%d", &integer1 ); // read

15 printf( "Enter second integer\n" ); // prompt
16     17                sum = integer1 + integer2; //
18     19
20                assign total to sum_printf( "Sum is
    scanf( "%d", &integer2, sum ); // print sum

21 } // end function main

```

```

Enter first integer
45
Enter second integer
72
Sum is 117

```

Fig. 2.5 | Addition program.

46 Chapter 2 Introduction to C Programming

The comment in lines 1–2 states the purpose of the program. As we stated earlier, every program begins execution with `main`. The left brace { (line 7) marks the beginning of the body of `main`, and the corresponding right brace } (line 21) marks the end of `main`.

Variables and Variable Definitions

Lines 8–10

```

int integer1; // first number to be entered by user
int integer2; // second number to be entered by user
int sum; // variable in which sum will be stored

```

are **definitions**. The names `integer1`, `integer2` and `sum` are the names of **variables**—locations in memory where values can be stored for use by a program. These definitions specify that variables `integer1`, `integer2` and `sum` are of type `int`, which means that they'll hold **integer** values, i.e., whole numbers such as 7, –11, 0, 31914 and the like.

All variables must be defined with a name and a data type *before* they can be used in a program. For readers using the Microsoft Visual C++ compiler, note that we're placing our variable definitions immediately after the left brace that begins the body of `main`. The C standard allows you to place each variable definition *anywhere* in `main` before that variable's first use in the code. Some compilers, such as GNU `gcc`, have

implemented this capability. We'll address this issue in more depth in later chapters.

The preceding definitions could have been combined into a single definition statement as follows:

```
int integer1, integer2, sum;
```

but that would have made it difficult to describe the variables with corresponding comments as we did in lines 8–10.

Identifiers and Case Sensitivity

A variable name in C is any valid **identifier**. An identifier is a series of characters consisting of letters, digits and underscores (_) that does *not* begin with a digit. C is **case sensitive**—uppercase and lowercase letters are *different* in C, so **a1** and **A1** are *different* identifiers.

Common Programming Error 2.2

*Using a capital letter where a lowercase letter should be used (for example, typing **Main** instead of **main**).*

Error-Prevention Tip 2.1

Avoid starting identifiers with the underscore character (_) to prevent conflicts with compiler-generated identifiers and standard library identifiers.

Good Programming Practice 2.5

Choosing meaningful variable names helps make a program self-documenting—that is, fewer comments are needed.

Good Programming Practice 2.6

The first letter of an identifier used as a simple variable name should be a lowercase letter. Later in the text we'll assign special significance to identifiers that begin with a capital letter and to identifiers that use all capital letters.

2.3 Another Simple C Program: Adding Two Integers 47

Good Programming Practice 2.7

*Multiple-word variable names can help make a program more readable. Separate the words with underscores as in **total_commissions**, or, if you run the words together, begin each word after the first with a capital letter as in **totalCommissions**. The latter style is preferred.*

Syntax Errors

We discussed what syntax errors are in Chapter 1. Recall that the Microsoft Visual C++ compiler requires variable definitions to be placed *after* the left brace of a function and *before* any executable statements. Therefore, in the program in Fig. 2.5, inserting the definition of **integer1** *after* the first **printf** would cause a syntax error in Visual C++.

Common Programming Error 2.3

Placing variable definitions among executable statements causes syntax errors in the Microsoft Visual C++ Compiler.

Prompting Messages

Line 12

```
printf( "Enter first integer\n" ); // prompt
```

displays the literal "Enter first integer" and positions the cursor to the beginning of the

next line. This message is called a **prompt** because it tells the user to take a specific action.

The scanf Function and Formatted Inputs

The next statement

```
scanf( "%d", &integer1 ); // read an integer
```

uses **scanf** (the “f” stands for “formatted”) to obtain a value from the user. The function reads from the *standard input*, which is usually the keyboard. This **scanf** has two arguments, “%d” and **&integer1**. The first, the **format control string**, indicates the *type* of data that should be entered by the user. The **%d conversion specifier** indicates that the data should be an integer (the letter **d** stands for “decimal integer”). The % in this context is treated by **scanf** (and **printf** as we’ll see) as a special character that begins a conversion specifier. The second argument of **scanf** begins with an ampersand (&—called the **address operator**—followed by the variable name. The &, when combined with the variable name, tells **scanf** the location (or address) in memory at which the variable **integer1** is stored. The computer then stores the value that the user enters for **integer1** at that location. The use of ampersand (&) is often confusing to novice programmers or to people who have programmed in other languages that do not require this notation. For now, just remember to precede each variable in every call to **scanf** with an ampersand. Some exceptions to this rule are discussed in Chapters 6 and 7. The use of the ampersand will become clear after we study *pointers* in Chapter 7.

Good Programming Practice 2.8

Place a space after each comma (,) to make programs more readable.

When the computer executes the preceding **scanf**, it waits for the user to enter a value for variable **integer1**. The user responds by typing an integer, then pressing the **Enter key** to send the number to the computer. The computer then assigns this number, or value, to

48 Chapter 2 Introduction to C Programming

the variable **integer1**. Any subsequent references to **integer1** in this program will use this same value. Functions **printf** and **scanf** facilitate interaction between the user and the computer. Because this interaction resembles a dialogue, it’s often called **interactive computing**. Line 15

```
printf( "Enter second integer\n" ); // prompt
```

displays the message **Enter second integer** on the screen, then positions the cursor to the beginning of the next line. This **printf** also prompts the user to take action. Line 16

```
scanf( "%d", &integer2 ); // read an integer
```

obtains a value for variable **integer2** from the user.

Assignment Statement

The **assignment statement** in line 18

```
sum = integer1 + integer2; // assign total to sum
```

calculates the total of variables **integer1** and **integer2** and assigns the result to variable **sum** using the assignment operator **=**. The statement is read as, “**sum gets** the value of **integer1 + integer2**.” Most calculations are performed in assignments. The **=** operator and the **+** operator are called *binary* operators because each has *two operands*. The **+** operator’s two operands are **integer1** and **integer2**. The **=** operator’s two operands are

sum and the value of the expression `integer1 + integer2`.

Good Programming Practice 2.9

Place spaces on either side of a binary operator. This makes the operator stand out and makes the program more readable.

Common Programming Error 2.4

A calculation in an assignment statement must be on the right side of the `=` operator. It's a compilation error to place a calculation on the left side of an assignment operator.

Printing with a Format Control String

Line 20

```
printf("Sum is %d\n", sum ); // print sum
```

calls function `printf` to print the literal `Sum is` followed by the numerical value of variable `sum` on the screen. This `printf` has two arguments, `"Sum is %d\n"` and `sum`. The first argument is the format control string. It contains some literal characters to be displayed, and it contains the conversion specifier `%d` indicating that an integer will be printed. The second argument specifies the value to be printed. Notice that the conversion specifier for an integer is the same in both `printf` and `scanf`—this is the case for most C data types.

Calculations in printf Statements

Calculations can also be performed inside `printf` statements. We could have combined the previous two statements into the statement

```
printf("Sum is %d\n", integer1 + integer2 );
```

The right brace, `}`, at line 21 indicates that the end of function `main` has been reached.

2.5

2.4 Memory Concepts 49

Common Programming Error

Forgetting to precede a variable in a `scanf` statement with an ampersand when that variable should, in fact, be preceded by an ampersand results in an execution-time error. On many systems, this causes a “segmentation fault” or “access violation.” Such an error occurs when a user’s program attempts to access a part of the computer’s memory to which it does not have access privileges. The precise cause of this error will be explained in Chapter 7.

Common Programming Error 2.6

Preceding a variable included in a `printf` statement with an ampersand when, in fact, that variable should not be preceded by an ampersand.

2.4 Memory Concepts

Variable names such as `integer1`, `integer2` and `sum` actually correspond to locations in the computer’s memory. Every variable has a name, a **type** and a **value**. In the addition program of Fig. 2.5, when the statement (line 13)

```
scanf("%d", &integer1 ); // read an integer
```

is executed, the value entered by the user is placed into a memory location to which the

name `integer1` has been assigned. Suppose the user enters the number 45 as the value for `integer1`. The computer will place 45 into location `integer1`, as shown in Fig. 2.6.

`integer1` 45

Fig. 2.6 | Memory location showing the name and value of a variable.

Whenever a value is placed in a memory location, the value *replaces* the previous value in that location; thus, this process is said to be **destructive**.

Returning to our addition program again, when the statement (line 16)

```
scanf( "%d", &integer2 ); // read an integer
```

executes, suppose the user enters the value 72. This value is placed into location `integer2`, and memory appears as in Fig. 2.7. These locations are not necessarily adjacent in memory. Once the program has obtained values for `integer1` and `integer2`, it adds these values and places the total into variable `sum`. The statement (line 18)

```
sum = integer1 + integer2; // assign total to sum
```

`integer1` 45 72
`integer2`

Fig. 2.7 | Memory locations after both variables are input.
50 Chapter 2 Introduction to C Programming

that performs the addition also *replaces* whatever value was stored in `sum`. This occurs when the calculated total of `integer1` and `integer2` is placed into location `sum` (destroying the value already in `sum`). After `sum` is calculated, memory appears as in Fig. 2.8. The values of `integer1` and `integer2` appear exactly as they did *before* they were used in the calculation. They were used, but not destroyed, as the computer performed the calculation. Thus, when a value is *read* from a memory location, the process is said to be **nondestructive**.

`integer1`

`integer2`

`sum`

Arithmetic in

C

45

72

117

Fig. 2.8 | Memory locations after a calculation. 2.5

Most C programs perform calculations using the C **arithmetic operators** (Fig. 2.9). Note the use of various special symbols not used in algebra. The **asterisk** (*) indicates

multipli cation and the percent sign (%) denotes the *remainder operator*, which is introduced below. In algebra, to multiply *a* times *b*, we simply place these single-letter variable names side by side, as in *ab*. In C, however, if we were to do this, *ab* would be interpreted as a single, two-letter name (or identifier). Therefore, C (and many other programming languages) require that multiplication be explicitly denoted by using the * operator, as in *a * b*. The arithmetic operators are all *binary* operators. For example, the expression *3 + 7* contains the binary operator *+* and the operands *3* and *7*.

Addition *+* *f + 7 f + 7* Subtraction *-* *p - c p - c* Multiplication *** *bm b * m x*

Division */* *x / y* or *x ÷ y x / y* Remainder *%* *r mod s r % s*

Fig. 2.9 | Arithmetic operators.

Integer Division and the Remainder Operator

Integer division yields an integer result. For example, the expression *7 / 4* evaluates to 1 and the expression *17 / 5* evaluates to 3. C provides the **remainder operator**, *%*, which yields the *remainder* after integer division. The remainder operator is an integer operator that can be used only with integer operands. The expression *x % y* yields the remainder af ter *x* is divided by *y*. Thus, *7 % 4* yields 3 and *17 % 5* yields 2. We'll discuss many interesting applications of the remainder operator.

Error 2.7
2.5 Arithmetic in C 51

Common Programming

An attempt to divide by zero is normally undefined on computer systems and generally re sults in a fatal error, i.e., an error that causes the program to terminate immediately with out having successfully performed its job. Nonfatal errors allow programs to run to completion, often producing incorrect results.

Arithmetic Expressions in Straight-Line Form

Arithmetic expressions in C must be written in **straight-line form** to facilitate entering programs into the computer. Thus, expressions such as “*a* divided by *b*” must be written as *a/b* so that all operators and operands appear in a straight line. The algebraic notation

b[−]

is generally not acceptable to compilers, although some special-purpose software packages do support more natural notation for complex mathematical expressions.

Parentheses for Grouping Subexpressions

Parentheses are used in C expressions in the same manner as in algebraic expressions. For example, to multiply *a* times the quantity *b + c* we write *a*(b+c)*.

Rules of Operator Precedence

C applies the operators in arithmetic expressions in a precise sequence determined by the following **rules of operator precedence**, which are generally the same as those in

algebra:

1. Operators in expressions contained within pairs of parentheses are evaluated first. Parentheses are said to be at the “highest level of precedence.” In cases of **nested**, or **embedded**, **parentheses**, such as

$((a + b) + c)$

the operators in the *innermost* pair of parentheses are applied first.

2. Multiplication, division and remainder operations are applied next. If an expression contains several multiplication, division and remainder operations, evaluation proceeds from left to right. Multiplication, division and remainder are said to be on the same level of precedence.
3. Addition and subtraction operations are evaluated next. If an expression contains several addition and subtraction operations, evaluation proceeds from left to right. Addition and subtraction also have the same level of precedence, which is lower than the precedence of the multiplication, division and remainder operations.
4. The assignment operator ($=$) is evaluated last.

The rules of operator precedence specify the order C uses to evaluate expressions.¹ When we say evaluation proceeds from left to right, we’re referring to the **associativity** of the operators. We’ll see that some operators associate from right to left. Figure 2.10 summarizes these rules of operator precedence for the operators we’ve seen so far.

1. We use simple examples to explain the order of evaluation of expressions. Subtle issues occur in more complex expressions that you’ll encounter later in the book. We’ll discuss these issues as they arise.